I still did not understand the role of pre keys here? How are they used, please explain

Yes — the easiest way to understand **prekeys** is to forget the rest of E2EE for a moment and focus on the problem they solve.

The problem prekeys solve

Imagine Bob is **offline**.

Alice wants to start a secure conversation with Bob at 2 PM:

```
Alice                          Bob
  |                             |
  | "Hey Bob"                   |
  |                             | OFFLINE
  |                             |
```

Alice needs some cryptographic information from Bob to establish the first secure session.

But Bob can't respond because he's offline.

So we need Bob to **prepare cryptographic material in advance**.

That's what **prekeys** are.

> **Prekeys are cryptographic keys that Bob generates in advance and uploads their public parts to the server, so someone can initiate an encrypted conversation with Bob even while Bob is offline.**

That's the entire purpose at a high level.

---

1. Without prekeys

Let's first see the problem.

Suppose Bob only has:

```
Bob:
    Private Key
    Public Key
```

Alice wants to start a conversation.

She asks the server:

```
Alice → Server:

"I want to talk securely with Bob.
Give me Bob's public key."
```

Server:

```
Server → Alice:

Here is Bob's public key.
```

That's actually enough for **some basic public-key encryption/key-agreement scenarios**.

But modern messaging needs something better.

In particular, we want to establish a fresh cryptographic session asynchronously, support stronger security properties, and avoid having a single long-lived public key serve as the only material available for every initiation.

That's where prekeys enter.

---

2. Bob prepares keys BEFORE Alice messages

When Bob's application is installed or initialized, it can generate a bunch of keys.

Think:

```
Bob's phone

Identity Key
    ↓
    BobIdentity

Prekey 1
    ↓
    Prekey1
```

```
Prekey 2
    ↓
    Prekey2

Prekey 3
    ↓
    Prekey3

Prekey 4
    ↓
    Prekey4

...
```

For each prekey there is a private/public pair:

```
Prekey 1:
    Private → stays on Bob's phone
    Public  → uploaded to server

Prekey 2:
    Private → stays on Bob's phone
    Public  → uploaded to server

Prekey 3:
    Private → stays on Bob's phone
    Public  → uploaded to server
```

So the server might have:

```
Bob
 ├── Identity Public Key
 ├── Prekey Public 1
 ├── Prekey Public 2
 ├── Prekey Public 3
 └── Prekey Public 4
```

But **not**:

```
Bob's Prekey Private 1
Bob's Prekey Private 2
Bob's Prekey Private 3
...
```

Those remain on Bob's device.

---

3. Now Alice wants to message Bob

Bob happens to be:

```
OFFLINE
```

Alice says:

```
Alice → Server:

"I want to establish a secure conversation with Bob."
```

The server can give Alice some of Bob's **public cryptographic material**.

Conceptually:

```
Server → Alice:

Bob Identity Public Key
Bob Signed Prekey
Bob One-Time Prekey
```

Don't worry about the different types yet. We'll get to them.

Alice now has enough information to construct the initial cryptographic handshake **without Bob being online**.

This is the key point.

---

4. Let's simplify it to one prekey

Forget all the complicated Signal Protocol terminology for a minute.

Suppose Bob has:

```
Bob:

Private Prekey = BP_private
Public Prekey  = BP_public
```

Bob uploads:

```
BP_public
```

to the server.

Alice downloads:

```
BP_public
```

Now Alice has:

```
Alice Private Key
+
Bob Public Prekey
```

She can perform a key agreement.

Conceptually:

```
Alice's private key
        +
Bob's public prekey
        ↓
   Key Agreement
        ↓
   Shared Secret
```

Bob can later use:

```
Bob's private prekey
        +
Alice's public information
        ↓
   Key Agreement
        ↓
   Same Shared Secret
```

So:

```
                Alice
                  |
                  | Alice private
                  |
                  | Bob public prekey
                  ↓
             Key Agreement
                  ↓
            Shared Secret
                  ↑
                  |
             Key Agreement
                  ↑
                  |
```

```
                  | Bob private prekey
                  |
                Bob
```

Alice doesn't have Bob's private prekey.

Bob doesn't have Alice's private key.

Yet both can derive the same secret.

---

5. Why is it called a "pre-key"?

Because Bob generated it **before the conversation began**.

That's it.

```
Before conversation:

Bob generates keys
       ↓
Prekeys
       ↓
Public parts uploaded
       ↓
WAIT



Later:

Alice wants to message Bob
       ↓
Alice obtains a prekey
       ↓
Uses it to establish session
       ↓
Bob eventually receives message
```

It's literally a key prepared **preemptively**.

---

6. But there's an even more interesting part: One-Time Prekeys

Suppose Bob has:

```
Prekey 1
Prekey 2
Prekey 3
Prekey 4
Prekey 5
```

Alice asks for one.

Server gives:

```
Prekey 3
```

Alice uses it.

Bob's device eventually processes that prekey.

Now:

```
Prekey 3 → USED
```

It should not be reused indefinitely.

The server can provide:

```
Prekey 4
```

to the next person/device session initiation.

This is why they're often called:

> **One-Time Prekeys (OTPKs)**

They provide fresh key material for asynchronous session establishment.

---

7. Why not just use Bob's normal public key?

Excellent question.

Suppose Bob has only:

```
Bob Identity Public Key
```

Alice uses it every time:

```
Alice → Bob
     ↓
Identity Public Key
     ↓
Session
```

Now Bob's long-term identity key is involved in every initiation.

Instead, the protocol can use additional ephemeral/prekey material.

Conceptually:

```
Long-term identity
       +
Prekey
       +
Alice ephemeral key
       ↓
Key agreement
       ↓
Initial session secrets
```

This gives the protocol more cryptographic flexibility and contributes to its security properties.

The **identity key answers "who is Bob?"**

The **prekey helps answer "how do I establish a fresh secure session with Bob right now, even though he's offline?"**

That distinction is extremely useful.

---

8. There are actually different kinds of keys

This is probably where my previous explanation made things confusing.

In a Signal-style system, don't think:

```
Bob has "a key"
```

Think:

```
Bob has a COLLECTION of cryptographic keys
```

Very roughly:

```
Bob
│
├── Identity Key Pair
│       ├── Identity Private
│       └── Identity Public
│
├── Signed Prekey Pair
│       ├── Signed Prekey Private
│       └── Signed Prekey Public
│
└── One-Time Prekey Pairs
        ├── OTPK 1 private/public
        ├── OTPK 2 private/public
        ├── OTPK 3 private/public
        └── ...
```

The exact Signal protocol details are more nuanced, but this mental model is good.

---

9. What is the Signed Prekey?

There's another important question:

> "If the server gives Alice some random public key claiming it's Bob's prekey, how does Alice know that it really belongs to Bob?"

That's why there's a **signed prekey**.

Bob has:

```
Bob Identity Private Key
```

He uses it to sign:

```
Bob Signed Prekey Public
```

Conceptually:

```
Bob Identity Private Key
          +
Signed Prekey Public
          ↓
       Signature
```

Bob uploads:

```
Identity Public Key
Signed Prekey Public
Signature
```

Alice receives them.

She can verify:

```
Signature
    ↓
Does Bob's identity key really authenticate this prekey?
```

So Alice gets confidence that:

```
This prekey belongs to Bob's cryptographic identity.
```

---

10. Now let's see the entire first-message flow

This is the part I recommend remembering.

Phase A — Bob registers

Bob's device creates:

```
Identity Key Pair
Signed Prekey Pair
Several One-Time Prekey Pairs
```

For example:

```
Bob's Device
————————————————————

Identity:
  Private Identity
  Public Identity

Signed Prekey:
  Private Signed Prekey
  Public Signed Prekey

One-Time Prekeys:
  Private OTPK 1
  Public OTPK 1

  Private OTPK 2
```

```
  Public OTPK 2

  Private OTPK 3
  Public OTPK 3
```

Phase B — Bob uploads public information

```
Bob
 |
 | Identity Public
 | Signed Prekey Public
 | Signature
 | OTPK Public 1
 | OTPK Public 2
 | OTPK Public 3
 |
 ↓
Server
```

Private keys remain:

```
Bob's phone
```

Phase C — Alice wants to start conversation

Bob is offline.

```
Alice                          Bob
  |                             |
  |                             | OFFLINE
  |                             |
```

Alice asks:

```
Alice → Server

"Give me Bob's key bundle."
```

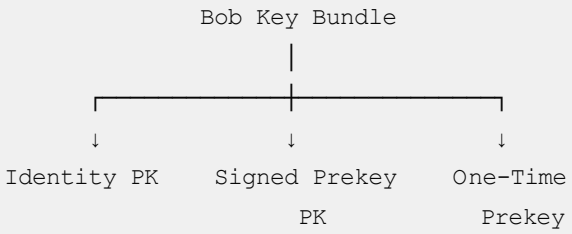
Phase D — Server returns Bob's public bundle

Conceptually:

```
Server → Alice

Bob Identity Public Key
Bob Signed Prekey Public
Signed Prekey Signature
Bob One-Time Prekey Public
```

For example:



---

11. Alice now creates her own ephemeral key

Alice's device generates a temporary key pair:

```
Alice Ephemeral Key Pair

       Private
          +
       Public
```

This is important because Alice doesn't just rely on her permanent identity key.

Now she has:

```
Alice Identity Private
Alice Ephemeral Private
```

and Bob's public material:

```
Bob Identity Public
Bob Signed Prekey Public
Bob One-Time Prekey Public
```

---

12. They perform several key agreements

This is where the actual Signal-style initial handshake becomes more sophisticated.

Very roughly, Alice performs several Diffie-Hellman calculations involving:

```
Alice Identity Key
Alice Ephemeral Key

Bob Identity Key
Bob Signed Prekey
Bob One-Time Prekey
```

Those results are combined through a key derivation function.

Conceptually:

```
        Alice's keys
             +
        Bob's public keys
             |
             ↓
      Several DH operations
             |
             ↓
       Combine results
             |
             ↓
           KDF
             |
             ↓
       Initial Secret
```

Bob can independently perform the corresponding calculations using his private keys.

Both arrive at compatible shared secret material.

---

13. And now Alice can send the first encrypted message

This is the beautiful part.

Bob is STILL offline.

Alice can now do:

```
"Hello Bob"
     ↓
Initial session key
     ↓
```

```
Encrypt
     ↓
Ciphertext
```

and send:

```
Alice
  |
  | ciphertext
  ↓
Server
```

Server stores it.

```
Server
  |
  | waiting...
  |
  X Bob offline
```

---

14. Bob eventually comes online

Bob:

```
Bob → Server

"I'm online."
```

Server:

```
"You have a message."
```

Then:

```
Server
   |
   | ciphertext
   ↓
Bob
```

Bob has the corresponding private cryptographic material.

He performs the corresponding derivation and gets the same session secret.

Then:

```
Ciphertext
    +
Session Key
    ↓
Decrypt
    ↓
"Hello Bob"
```

---

15. So what EXACTLY did the prekey accomplish?

This is the most important answer.

Without the prekey concept:

```
Alice wants secure session
        ↓
Needs Bob's participation
        ↓
Bob is offline
        ↓
Problem
```

With prekeys:

```
Bob prepares cryptographic material
        ↓
Uploads public portions
        ↓
Bob goes offline
        ↓
Alice gets Bob's prekey
        ↓
Alice establishes initial session
        ↓
Alice encrypts first message
        ↓
Server stores ciphertext
        ↓
Bob comes online later
        ↓
Bob decrypts
```

So:

> **The prekey is what allows the cryptographic handshake to be initiated asynchronously.**

That's the key sentence.

---

16. One analogy that makes this click

Imagine Bob owns a secure mailbox.

Before going offline, Bob puts several **pre-authorized locks** at the post office.

```
Bob
 |
 | prepares locks
 ↓
Post Office
```

Alice comes later:

```
Alice → Post Office

"I want to send Bob something securely."
```

The post office can give Alice one of Bob's prepared locks.

Alice uses it to create the secure package.

Bob doesn't need to be physically present.

Later:

```
Bob comes back
     ↓
gets package
     ↓
uses corresponding private key
     ↓
opens it
```

The post office never learns what's inside.

That's roughly the role of prekeys.

---

17. The three things to keep separate

This is the mental model I'd recommend:

Identity key

Answers:

> **"Who is Bob?"**

```
Bob Identity Key
```

Long-lived cryptographic identity.

Signed prekey

Answers roughly:

> **"Here's a current public key authenticated by Bob's identity that can participate in establishing a session."**

```
Identity Key
      ↓ signs
Signed Prekey
```

It can remain available for a period of time and be rotated.

One-time prekey

Answers roughly:

> **"Here's additional fresh key material that can be consumed for an asynchronous session initiation."**

```
OTPK 1 → use → discard
OTPK 2 → use → discard
OTPK 3 → use → discard
```

---

18. Finally, don't confuse prekeys with message keys

This distinction is **very important**.

Prekeys are primarily involved in **starting/bootstrapping the secure session**.

They are **not** the key used to encrypt every chat message.

Think:

```
START CONVERSATION
        |
        ↓
Prekeys / DH
        |
        ↓
Initial shared secret
        |
        ↓
Double Ratchet
        |
        ↓
Message Key 1
Message Key 2
Message Key 3
Message Key 4
    ...
        |
        ↓
Actual messages
```

So:

Prekeys → help establish the initial secure state.

Ratchet/message keys → protect the actual ongoing messages.

That separation is the key to understanding the whole Signal-style E2EE architecture.

How many pre keys are usually generated? If the user is offline and 100s of users messages, what happens?

Exactly — **this is the scalability question with prekeys**. If Bob is offline and 500 people try to start conversations with him, you might wonder: *"Does Bob need 500 prekeys sitting on the server?"*

The answer is **not necessarily**, and there are some important nuances.

1. How many prekeys are generated?

There isn't one universal number. A Signal-style client typically maintains a **pool** of one-time prekeys.

For example, imagine Bob's phone generates:

```
OTPK-001
OTPK-002
OTPK-003
...
OTPK-100
```

and uploads the **public portions** to the server.

The server might have:

```
Bob
 ├── Identity key
 ├── Signed prekey
 │
 └── One-time prekeys
      ├── 001
      ├── 002
      ├── 003
      ├── ...
      └── 100
```

The exact pool size and replenishment policy are implementation-dependent. The important concept is that **the client keeps replenishing the pool**, rather than generating an infinite number in advance.

---

2. Now imagine Bob is offline

Suppose Bob has:

```
100 one-time prekeys
```

And then 100 different users want to initiate conversations:

```
Alice   ──┐
John     ─┤
Sarah    ─┤
Mike     ─┤
...       ├──> Bob
...       │
User 100 ─┘
```

The server can give different one-time prekeys to different initial session establishments:

```
Alice → OTPK-001
John  → OTPK-002
Sarah → OTPK-003
Mike  → OTPK-004
...
```

Conceptually:

```
Bob's prekey pool

001 ── consumed
002 ── consumed
003 ── consumed
004 ── consumed
...
100 ── consumed
```

The private portions remain on Bob's device.

---

3. What if 500 people message Bob?

This is where the answer becomes interesting.

Suppose Bob only has:

```
100 one-time prekeys
```

but:

```
500 people
```

try to start new sessions while he's offline.

You **don't need 500 one-time prekeys to make 500 messages possible**.

One-time prekeys are additional cryptographic material for **session establishment**, not a ticket required for every message.

Once Alice has established a session:

```
Alice ↔ Bob
```

Alice can send:

```
Message 1
Message 2
Message 3
Message 4
...
Message 500
```

using the resulting session/ratchet state.

Those messages do **not** consume additional prekeys.

---

4. So what happens after the 100 prekeys are consumed?

The server can essentially say:

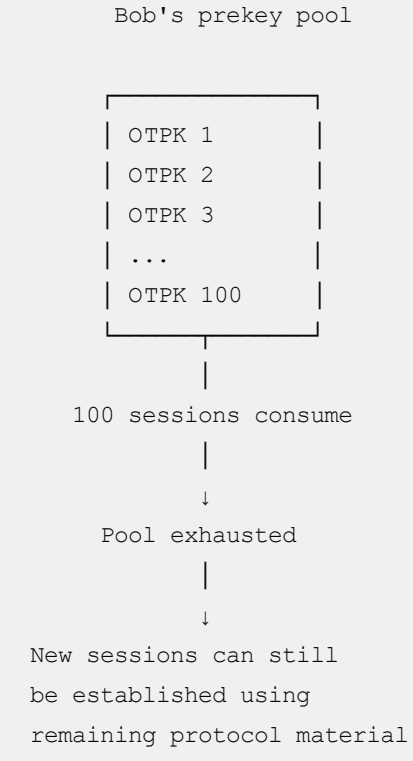
> "I don't have another one-time prekey available for this new session."

But this doesn't necessarily mean:

> "Bob cannot receive messages."

There is still other public key material, such as the **signed prekey**, that can be used for session establishment.

So conceptually:



The **one-time prekey is an extra security component**, not the fundamental mechanism that makes messaging possible.

---

5. Bob eventually comes online

This is where replenishment happens.

Bob's client connects:

```
Bob's phone
     |
     | "I'm online"
     ↓
Server
```

The client can determine that its one-time-prekey inventory is low.

For example:

```
Before:

100 OTPKs

After many sessions:

5 OTPKs
```

Bob's client generates more:

```
OTPK-101
OTPK-102
OTPK-103
...
OTPK-200
```

and uploads their public portions.

So:

```
             Bob's device
                  |
                  | generate
                  ↓
            New OTPKs
                  |
                  | public portions
```

```
                ↓
              Server
```

Now the pool is healthy again.

---

6. This is similar to a connection pool

Since you're a backend engineer, here's a useful analogy.

Think of one-time prekeys as a **pool of pre-created resources**.

```
Connection Pool:

Connection 1
Connection 2
Connection 3
...
Connection 100
```

Clients consume them.

When the pool gets low:

```
5 remaining
     ↓
generate more
     ↓
pool back to 100
```

Prekeys work somewhat similarly:

```
Prekey Pool:

OTPK 1
OTPK 2
...
OTPK 100

       ↓
session establishment
       ↓
consume
       ↓
pool decreases
       ↓
replenish
```

But remember: **prekeys aren't network connections**; this is only an analogy for understanding the pool/replenishment behavior.

---

7. Now let's consider your "hundreds of users" scenario properly

Suppose:

```
Bob = offline for 24 hours

1,000 users
   ↓
try to message Bob
```

There are actually **two separate problems**:

Problem A — Initial session establishment

For users who have **never established a session with Bob**, the protocol needs appropriate cryptographic material to bootstrap those sessions.

Some may consume one-time prekeys.

Problem B — Message delivery

Once a session already exists:

```
Alice ↔ Bob
```

Alice can send many messages without consuming a new prekey for every message.

The server simply queues:

```
Alice → ciphertext 1
Alice → ciphertext 2
Alice → ciphertext 3
...
```

until Bob comes online.

---

8. This distinction is extremely important

Imagine Bob has already chatted with Alice yesterday.

They already have:

```
Alice ↔ Bob
       |
       ↓
Established session
       |
       ↓
Double Ratchet state
```

Bob goes offline.

Alice sends:

```
"Hey"
"Are you free?"
"Call me"
"Important question"
```

She **doesn't need four new prekeys**.

Instead:

```
Existing Alice-Bob session
          |
          ↓
     Ratchet forward
          |
     ┌────┼────┬────┐
     ↓    ↓    ↓    ↓
    K1   K2   K3   K4
     ↓    ↓    ↓    ↓
    M1   M2   M3   M4
```

The server queues those ciphertexts.

---

9. What if Alice has NEVER talked to Bob?

Now it's different.

Alice needs to establish a new session:

```
Alice ————————> Bob
     new session
```

Bob's server-side key bundle provides the public material needed for asynchronous setup.

If a one-time prekey is available:

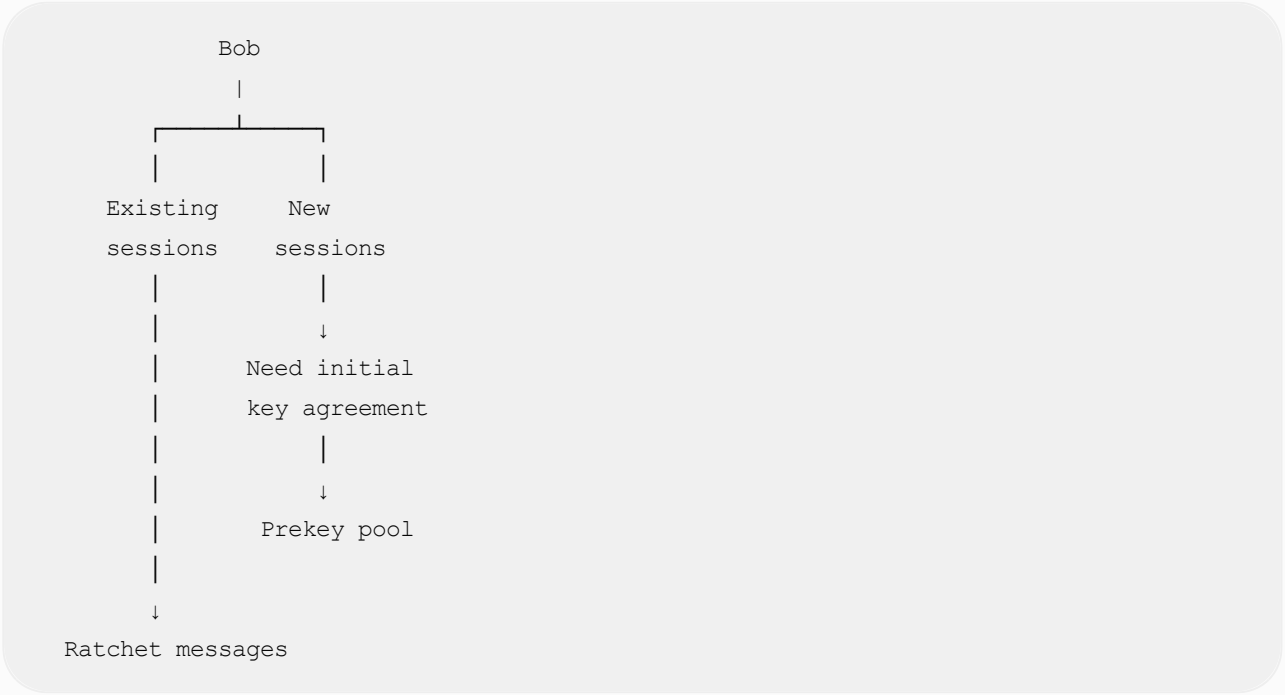
```
Alice
  |
  | Bob's public OTPK
  ↓
Initial key agreement
  ↓
New session
```

That OTPK is then considered consumed.

---

10. What if 1,000 NEW users contact Bob while offline?

Conceptually:



If the OTPK pool isn't sufficient, the protocol can still establish sessions using the other available key material; however, **the exact behavior depends on the protocol/version and implementation**.

The one-time prekey primarily provides an **additional layer of asynchronous, fresh key material**.

---

11. Why not just generate 1 million prekeys?

You could imagine:

```
Bob generates:

1,000,000 prekeys
```

But that's unnecessary.

Because:

Storage

You now have huge amounts of cryptographic state.

Management

The client needs to track:

```
unused
used
uploaded
acknowledged
```

Server storage

The server needs to maintain the public pool.

Device resources

You don't want to unnecessarily generate and manage huge numbers of keys.

Instead:

```
Small pool
   ↓
Consume
   ↓
Monitor
   ↓
Replenish
```

is much more sensible.

---

12. And there's an important nuance about "offline"

There are actually **two different meanings of offline** here.

Bob's app isn't currently connected

```
Bob's phone
    X
Server
```

But the server still has Bob's public key bundle.

Alice can therefore initiate a session.

Bob hasn't used the app for a very long time

Now things become more complicated.

The server may eventually have:

```
few/no OTPKs
```

and the client's key material/state may need to be refreshed when Bob returns.

Different messaging systems handle this differently.

---

13. Think of prekeys as "permission slips"

Here's perhaps the simplest analogy.

Bob leaves **100 sealed envelopes** at the post office.

```
Bob
 |
 | 100 sealed envelopes
 ↓
Post Office
```

Alice comes:

```
Alice:
"I want to start a secure conversation with Bob."
```

Post office gives Alice:

```
Envelope #37
```

Alice uses the information from that envelope to establish her secure session.

Later:

```
Envelope #37 → consumed
```

Bob doesn't need to be at the post office.

But if:

```
100 envelopes
```

are consumed, the post office can continue using the other protocol mechanisms for new sessions, and when Bob's device is available it can replenish the pool.

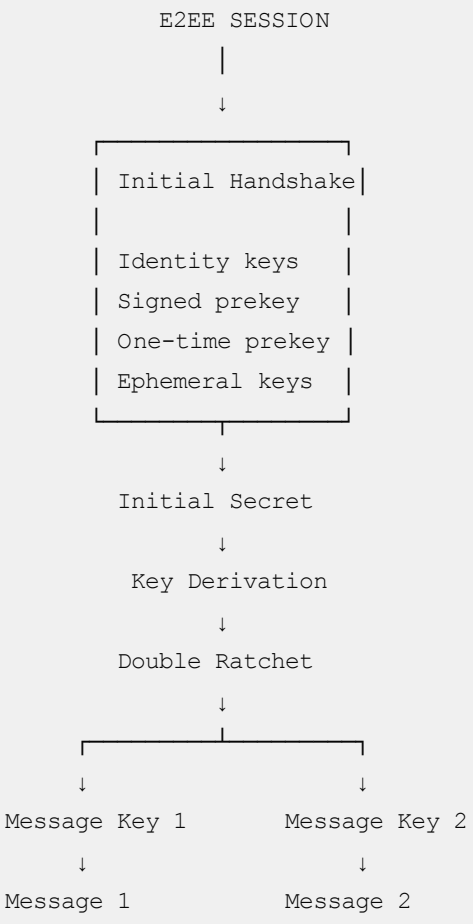
---

14. One correction to the mental model from my previous answer

I want to make this distinction very clear because it's easy to misunderstand:

Prekeys are NOT "the encryption keys for the conversation."

Instead:



So:

> **Prekeys help you get from "we don't have a session" → "we have a secure session."**

After that, the **ratchet takes over**.

The whole thing in one example

Bob is offline.

```
Bob's server-side pool:

OTPK1
OTPK2
OTPK3
...
OTPK100
```

Alice contacts Bob

```
Alice → Server:
"I want Bob's key bundle."

Server → Alice:
Identity Public Key
Signed Prekey
OTPK1
```

Alice establishes a session.

```
OTPK1 → consumed
```

Alice sends:

```
M1 → encrypted with message key 1
M2 → encrypted with message key 2
M3 → encrypted with message key 3
```

Server queues:

```
Queue for Bob:

Ciphertext M1
```

```
Ciphertext M2
```

Then 500 other users can do similar **initial session establishment**, subject to the available prekey pool and protocol behavior.

When Bob comes online:

```
Bob
 ↓
receives queued ciphertexts
 ↓
decrypts using his session states
 ↓
messages appear
```

And his client can replenish the one-time prekey pool.

The one sentence to remember

Prekeys are a server-published pool of cryptographic material that lets other devices bootstrap secure sessions with an offline device; they are consumed during session setup, not once per message.

So, if I have 1000 users in my whatsapp, it's possible to have 1000 private prekeys in my device and 1000 public pre keys in whatsapp db?

Not quite. **This is the key distinction:** prekeys are generally **not generated one-per-contact/user**.

If you have 1,000 WhatsApp contacts, you don't normally have:

```
Your phone
│
├── Private Prekey for Alice
├── Private Prekey for Bob
├── Private Prekey for Charlie
├── ...
└── Private Prekey for User 1000
```

Instead, think of them as a **pool of one-time prekeys belonging to your device**:

```
Your phone

Identity Key
    │
    ├── Private Identity Key
    └── Public Identity Key

Signed Prekey
    │
    ├── Private Signed Prekey
    └── Public Signed Prekey

One-Time Prekey Pool
    │
    ├── Private OTPK 1
    ├── Private OTPK 2
    ├── Private OTPK 3
    ├── ...
    └── Private OTPK N
```

And the server has the corresponding **public** one-time prekeys:

```
WhatsApp Server

Your Device
│
├── Identity Public Key
├── Signed Prekey Public Key
│
└── One-Time Prekeys
      ├── Public OTPK 1
      ├── Public OTPK 2
      ├── Public OTPK 3
      └── ...
```

The important part

The mapping is **not**:

```
1 contact = 1 prekey
```

It's more like:

```
1 device
   ↓
1 pool of prekeys
   ↓
different new session establishments consume them
```

For example:

```
You have 1,000 contacts

          Your Device
              |
              |
       100 OTPKs available
              |
       ┌──────┼──────┐
       ↓      ↓      ↓
    Alice    Bob    Charlie
       |      |       |
       ↓      ↓       ↓
    OTPK 1  OTPK 2  OTPK 3
```

Alice, Bob, and Charlie don't each permanently "own" one of your prekeys.

A prekey is consumed when it's used to **bootstrap a new cryptographic session**.

So could you have 1,000 private prekeys?

Yes, technically. A device could have a pool of 1,000 one-time prekey pairs.

Then you'd have roughly:

```
Your phone                    Server

Private OTPK 1  <----------->  Public OTPK 1
Private OTPK 2  <----------->  Public OTPK 2
...
Private OTPK 1000 <----------> Public OTPK 1000
```

But that **doesn't mean you need 1,000 because you have 1,000 contacts**.

You might have:

```
1,000 contacts
        +
only 100 OTPKs
```

and that's perfectly reasonable.

Why?

Because most of those 1,000 contacts may already have an established session with you, or may never initiate a new session with your device.

Think about it this way

Suppose you have:

```
1,000 contacts
100 one-time prekeys
```

Over some period:

```
Alice starts new session → OTPK #1 consumed
Bob starts new session   → OTPK #2 consumed
John starts new session  → OTPK #3 consumed
...
```

After 90 new session establishments:

```
100 → 10 remaining
```

Your client can generate more:

```
10 remaining
    ↓
generate 90 new OTPKs
    ↓
100 available again
```

So it's a **replenished pool**, not a contact database.

One more subtle point

A WhatsApp-style real system is more complicated than this simplified model because users can have **multiple devices**, and each device has its own cryptographic state.

So if you have:

```
Phone
Laptop
Tablet
```

you should mentally think:

```
Your Account
│
├── Phone
│    ├── Identity keys
│    ├── Signed prekey
│    └── OTPK pool
│
├── Laptop
│    ├── Identity keys
│    ├── Signed prekey
│    └── OTPK pool
│
└── Tablet
     ├── Identity keys
     ├── Signed prekey
     └── OTPK pool
```

Each device participates in its own cryptographic sessions.

So the really useful mental model is:

> **Contacts are not allocated prekeys. Devices maintain pools of prekeys, and those prekeys are consumed when new secure sessions are bootstrapped.**

And this also explains why **1000 contacts ≠ 1000 prekeys**.

so, once the session is created, prekeys are destroyed in sever?

For a one-time prekey, yes — roughly speaking. But there's an important distinction between **the server deleting its public copy** and **the private key being destroyed on the recipient's device**.

Let's walk through it.

1. Bob has a pool

Bob's device creates:

```
OTPK #1
OTPK #2
OTPK #3
```

```
...
```

Each has:

```
Private key → Bob's device
Public key  → Server
```

So:

```
Bob's phone                 Server

Private OTPK #1  <-------->  Public OTPK #1
Private OTPK #2  <-------->  Public OTPK #2
Private OTPK #3  <-------->  Public OTPK #3
```

---

2. Alice wants to start a new session with Bob

Alice asks the server for Bob's key bundle.

The server gives Alice:

```
Bob Identity Public Key
Bob Signed Prekey Public Key
Bob One-Time Prekey #1 Public Key
```

At this point, **OTPK #1 has been allocated/claimed for that session initiation**.

The server should not hand that same one-time prekey out to another new session.

Conceptually:

```
Server

OTPK #1 → RESERVED/CONSUMED
OTPK #2 → AVAILABLE
OTPK #3 → AVAILABLE
...
```

---

3. Alice establishes the session

Alice uses the public OTPK as part of the initial key agreement.

Bob's device eventually uses the corresponding private OTPK:

```
Alice                          Bob

Alice ephemeral
     +
Bob public OTPK
     |
     ↓
 Key Agreement
     |
     |                    Bob private OTPK
     |                         +
     |                    Alice public info
     |                         |
     ↓                         ↓
       Same initial secret
```

Now the session can be established.

---

4. What happens to OTPK #1?

This is the important part.

The **one-time prekey is not supposed to be reused for another independent session**.

So conceptually:

```
OTPK #1

Server:
       don't give it to another session

Bob's device:
       don't reuse the private key for another session
```

The exact deletion timing and implementation details depend on the protocol/client, but the security model is that a one-time prekey is **single-use**.

---

5. But the server doesn't destroy ALL prekeys

Suppose Bob had:

```
OTPK #1
OTPK #2
OTPK #3
```

```
...
```

Alice uses #1.

You now have:

```
Server

#1   → consumed
#2   → available
#3   → available
...
#100 → available
```

So the server doesn't wipe Bob's entire prekey pool.

Only the particular one-time prekey that was allocated/used is no longer available for another new session.

---

6. And this is why the pool gets replenished

Eventually:

```
Before:

#1 #2 #3 #4 ... #100
```

Lots of new sessions happen:

```
#1 → consumed
#2 → consumed
#3 → consumed
...
#90 → consumed
```

Now:

```
10 remaining
```

Bob's client can generate more:

```
#101
#102
#103
...
```

and upload their public portions.

So you get:

```
       Consume
          ↓
┌──────────────────┐
| Prekey Pool      |
|                  |
| 100 → 90 → 50    |
|           ↓      |
|       replenish  |
|           ↓      |
|         100      |
└──────────────────┘
```

One subtle but important correction

Don't think of it as:

> "Alice creates a session → server deletes the prekey."

It's more accurate to think:

> **"Alice obtains/claims a one-time prekey → that prekey is marked unavailable for another session → the corresponding private key is used by Bob's device during session establishment and should not be reused."**

And **the prekey itself is not used for the subsequent chat messages**.

After that:

```
One-Time Prekey
       ↓
Initial Key Agreement
       ↓
Session established
       ↓
Double Ratchet
       ↓
Message Key 1
Message Key 2
Message Key 3
Message Key 4
...
```

That's the complete lifecycle.

Prekey = bootstrap material.
Session/ratchet keys = actual ongoing conversation encryption.

Got it.

Great. The **prekey → session establishment → ratchet/message keys** distinction is the key piece.

What are epemeral keys

An **ephemeral key** is a cryptographic key that is **generated for temporary use**, usually for one session or one key exchange, and then discarded.

The easiest way to understand it is by comparing it with the keys we just discussed.

1. Three different kinds of keys

Imagine Alice wants to start an E2E conversation with Bob.

```
Bob's device

Identity Key
  ├── Private → stays forever-ish on Bob's device
  └── Public  → can be shared

Signed Prekey
  ├── Private → stays on Bob's device
  └── Public  → uploaded to server

One-Time Prekey
  ├── Private → stays on Bob's device
  └── Public  → uploaded to server
```

Now Alice also generates an **ephemeral key pair**:

```
Alice's device

Ephemeral Key
  ├── Private → temporary
  └── Public  → sent as part of session setup
```

The important distinction:

> **Prekeys are generated ahead of time by Bob. Ephemeral keys are typically generated by Alice when she starts establishing a session.**

---

2. Why do we need an ephemeral key?

Suppose Alice wants to establish a shared secret with Bob.

Alice creates:

```
Alice
Ephemeral private key
       +
Bob's public key material
       ↓
Shared secret
```

Bob can independently calculate the same secret using his corresponding private keys.

So:

```
             Key Exchange

Alice                              Bob
-----                              ---
Ephemeral private ──────────────────> Ephemeral public

Bob's public keys <────────────────── Bob's private keys

             ↓
        Shared Secret
             ↓
       Session established
```

The actual mathematics is based on **Diffie-Hellman-style key agreement**.

---

3. Why call it "ephemeral"?

Because Alice doesn't want to keep using that same key forever.

For example:

```
10:00 AM
Alice generates Ephemeral Key A
        ↓
```

```
Establishes session
        ↓
Ephemeral Key A eventually discarded


11:00 AM
Alice needs another session
        ↓
Generates Ephemeral Key B
```

So:

```
Ephemeral A → temporary
Ephemeral B → temporary
Ephemeral C → temporary
```

They're deliberately short-lived.

---

4. Ephemeral ≠ One-Time Prekey

This is an important distinction because the names sound similar.

One-Time Prekey

Generated **beforehand** by Bob:

```
Bob:
OTPK1
OTPK2
OTPK3
...
OTPK100
```

Bob uploads the public portions to the server.

Alice can consume one when starting a session.

Ephemeral key

Generated **when needed** by Alice:

```
Alice starts session
       ↓
Generate ephemeral key pair
       ↓
Use it in key agreement
       ↓
Derive shared secret
```

```
        ↓
Eventually discard it
```

So the mental model is:

```
                Session Setup

Bob                          Alice
---                          -----

Identity Key
Signed Prekey
One-Time Prekey
      |
      |  public material
      └──────────────────────────>
                                Generate
                                Ephemeral Key
                                     |
                                     ↓
                                Key Agreement
                                     |
                                     ↓
                                Shared Secret
```

---

5. Why is this useful for security?

This is where ephemeral keys become really important.

Suppose Alice used the **same permanent private key** for every conversation.

If that private key were somehow compromised, an attacker might gain much more information about past/future sessions.

With ephemeral keys, you can get **forward secrecy**.

Very simplified:

```
Session 1 → Ephemeral A → Message keys A
Session 2 → Ephemeral B → Message keys B
Session 3 → Ephemeral C → Message keys C
```

Compromise of one piece of key material doesn't automatically expose every other session.

And in Signal-style messaging, this gets even stronger because the **Double Ratchet** continually derives new keys:

```
Session established
       ↓
Root Key
       ↓
Chain Key
       ↓
Message Key 1
       ↓
Message Key 2
       ↓
Message Key 3
       ↓
Message Key 4
       ...
```

So there are actually **multiple layers of temporary cryptographic material**.

One simple analogy

Think of:

- **Identity key** = your permanent ID/passport
- **Prekeys** = spare sealed tickets you prepare in advance
- **Ephemeral key** = a temporary ticket you create specifically for today's interaction
- **Session key/message keys** = temporary keys used to actually lock the messages

The key idea:

> **An ephemeral key is a short-lived key pair created for a particular cryptographic operation/session, rather than being a long-term identity key.**

Now, explain step by step how connection happen to messaging happens both first time and next time.

Yes. Let's trace the **complete lifecycle**, because the important thing is to separate:

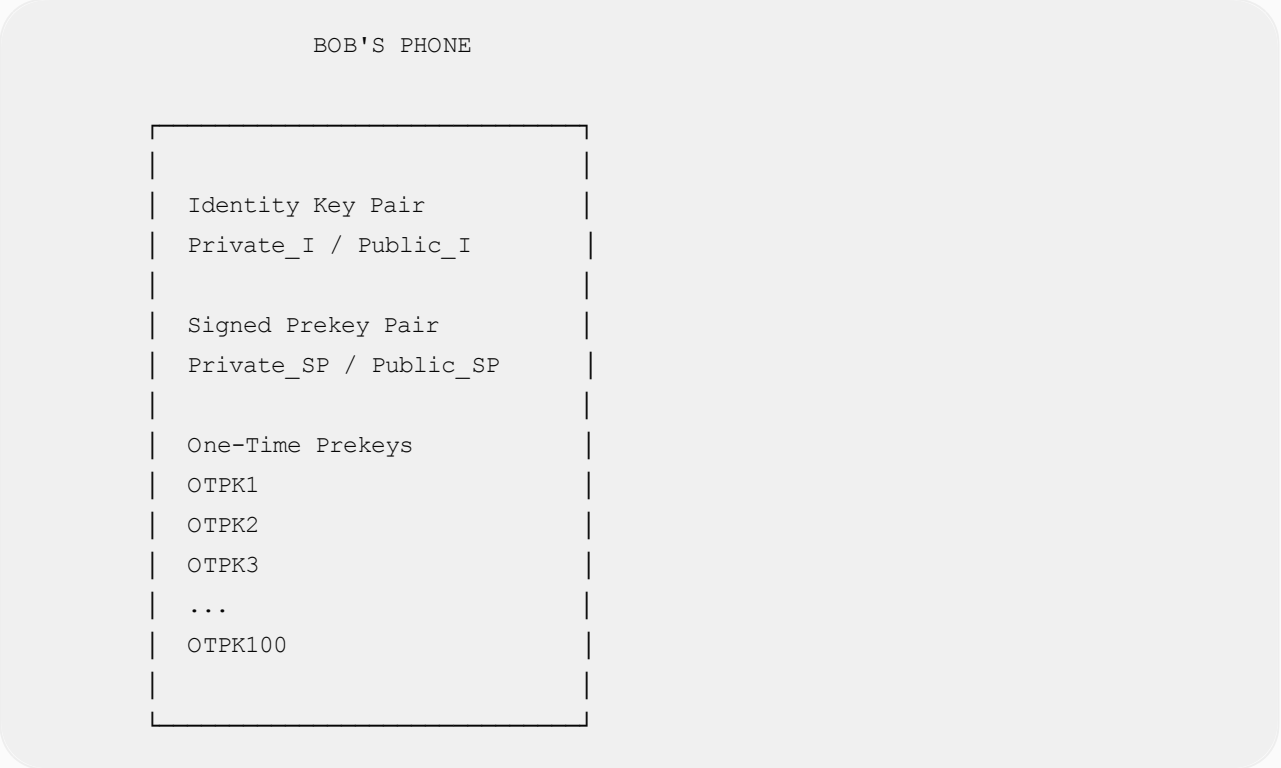
1. **First time Alice starts chatting with Bob**
2. **Subsequent messages in the same conversation**
3. **What happens if Bob is offline**

I'll use a Signal-style E2EE system as the model. WhatsApp uses Signal-derived cryptography, although its exact implementation details are proprietary.

---

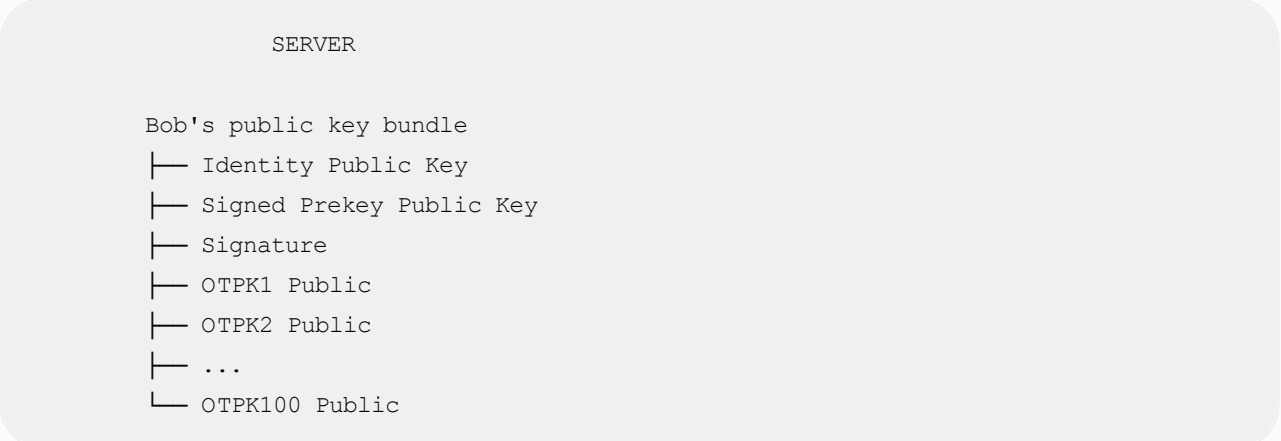
1. Before Alice ever messages Bob

First, Bob's phone prepares cryptographic material.



The **private keys never leave Bob's device**.

Bob uploads public material to the server:



The server doesn't have Bob's private keys.

---

2. Alice wants to send Bob her FIRST message

Alice opens:

> "Bob"

and types:

> Hello Bob!

At this point, Alice and Bob **do not yet have a shared session key**.

So Alice first needs to establish one.

---

3. Alice asks the server for Bob's key bundle

Alice's phone says:

```
Alice → Server:

"Give me Bob's public cryptographic bundle"
```

Server returns something like:

```
Bob:
    Identity Public Key
    Signed Prekey Public Key
    Signature
    OTPK37 Public Key
```

Notice:

```
Alice gets PUBLIC keys.
Alice does NOT get Bob's private keys.
```

---

4. Alice verifies Bob's signed prekey

The signed prekey is signed using Bob's identity key.

Conceptually:

```
Bob

Identity Private Key
        |
        | signs
        ↓
Signed Prekey
```

Alice verifies:

```
Signed Prekey
      ↓
Signature verification
      ↓
Bob's Identity Public Key
      ↓
Valid?
```

This helps Alice establish that the prekey belongs to Bob's identity.

---

5. Alice generates an ephemeral key

This is the part we were just discussing.

Alice creates a new temporary key pair:

```
Alice

Ephemeral Private Key
Ephemeral Public Key
```

For example:

```
Alice's phone

Identity Key
     +
Ephemeral Key
```

The ephemeral key is not Alice's permanent identity.

It's temporary cryptographic material used for establishing the session.

---

6. Alice performs key agreement

Now Alice has:

```
Alice:
    Her private keys
    Her ephemeral private key

Bob:
    Identity public key
```

```
    Signed prekey public key
```

Using a series of Diffie-Hellman operations, Alice derives an initial shared secret.

Conceptually:

```
Alice                              Bob
-----                              ---

Alice ephemeral private
        +
Bob public keys
        |
        ↓
   Shared Secret
```

Bob will later perform the corresponding calculations using his private keys.

The important property is:

```
Alice calculates → SECRET X
Bob calculates   → SECRET X

Server calculates →
```

The server sees public cryptographic material but cannot derive the session secret.

---

7. Alice sends the first encrypted message

Now Alice has enough information to establish the session.

She encrypts:

```
"Hello Bob!"
```

using derived symmetric encryption keys.

Something conceptually like:

```
Plaintext

"Hello Bob!"
     |
     ↓
Encryption
     |
     ↓
```

```
Ciphertext

8f72a9c1...x7ab29...
```

Alice sends to the server:

```
Alice → Server

{
    sender: Alice,
    recipient: Bob,
    ephemeral_public_key: ...,
    ciphertext: "8f72a9..."
}
```

The exact protocol message is more complicated, but this is the mental model.

---

8. Bob may be OFFLINE

This is where the architecture becomes interesting.

Suppose Bob's phone is switched off.

The server receives:

```
Alice
  │
  │ encrypted message
  ↓
Server
  │
  │ Bob offline
  ↓
Store ciphertext
```

The server stores the encrypted message.

It **cannot read**:

```
"Hello Bob!"
```

because it only has:

```
8f72a9c1...
```

---

9. Bob comes online

Bob opens WhatsApp/Signal/etc.

His phone connects:

```
Bob → Server

"I'm online."
"Give me my pending messages."
```

Server sends:

```
Server → Bob

Encrypted message
Ephemeral public key
Other protocol metadata
```

---

10. Bob derives the SAME shared secret

Bob now has the corresponding private cryptographic material.

He performs the matching key-agreement operations:

```
Bob's private keys
       +
Alice's ephemeral public key
       ↓
Same shared secret
```

So:

```
Alice calculated:

        SECRET = X


Bob calculated:

        SECRET = X
```

The server still doesn't know `X`.

---

11. Bob decrypts the message

Now Bob can do:

```
Ciphertext
    |
    ↓
Session/Message Key
    |
    ↓
Decryption
    |
    ↓
"Hello Bob!"
```

The first E2E session has been established.

---

12. What happens to the one-time prekey?

Remember Bob had:

```
OTPK1
OTPK2
OTPK3
...
OTPK100
```

Alice used:

```
OTPK37
```

That particular one-time prekey should no longer be used for another independent session.

Conceptually:

```
Before:

OTPK1   available
OTPK2   available
...
OTPK37  available
...
OTPK100 available


After:
```

```
OTPK1   available
OTPK2   available
...
OTPK37  USED
...
OTPK100 available
```

Bob's client can replenish the pool later.

---

13. Now Alice sends the SECOND message

This is the really important part.

Alice types:

> How are you?

Does Alice need another prekey?

No.

Does Alice need to ask the server for Bob's key bundle again?

Normally, no.

The session already exists.

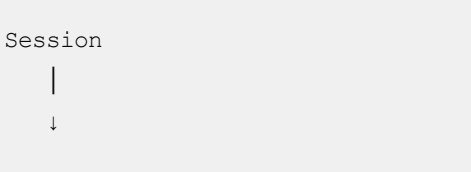
They now have ongoing cryptographic state.

---

14. Double Ratchet takes over

The session has key material from which new message keys are derived.

Simplified:



```
Root Key
   │
   ↓
Chain Key
   │
   ├──→ Message Key 1
   │
   ├──→ Message Key 2
   │
   ├──→ Message Key 3
   │
   └──→ Message Key 4
```

Alice's second message might use:

```
Message Key 2
```

Third message:

```
Message Key 3
```

Fourth:

```
Message Key 4
```

etc.

So:

```
Message 1 → Key 1
Message 2 → Key 2
Message 3 → Key 3
Message 4 → Key 4
```

This is one of the major security properties of the Signal-style protocol.

---

15. Alice sends the second message

```
"How are you?"
       │
       ↓
Message Key 2
       │
       ↓
Encryption
       │
```

```
       ↓
Ciphertext
       |
       ↓
Server
       |
       ↓
Bob
```

The server still sees only encrypted data.

---

16. What if Bob is offline again?

Exactly the same transport mechanism works.

```
Alice
  |
  | encrypt
  ↓
Server
  |
  | Bob offline
  ↓
Store ciphertext
```

Bob comes online later:

```
Server
  |
  | encrypted messages
  ↓
Bob
  |
  ↓
Decrypt locally
```

And importantly:

> **Alice does not need a new prekey just because Bob was offline.**

The existing session allows Alice to continue sending encrypted messages.

---

17. So what is the difference between FIRST and NEXT message?

This is the most important mental model.

First message

You need to **establish a session**:

```
Alice
  │
  ├── Get Bob's public key bundle
  │
  ├── Generate ephemeral key
  │
  ├── Use Bob's prekeys
  │
  ├── Perform key agreement
  │
  ├── Derive initial secret
  │
  └── Encrypt first message
          │
          ↓
        Server
          │
          ↓
         Bob
```

Next messages

The session already exists:

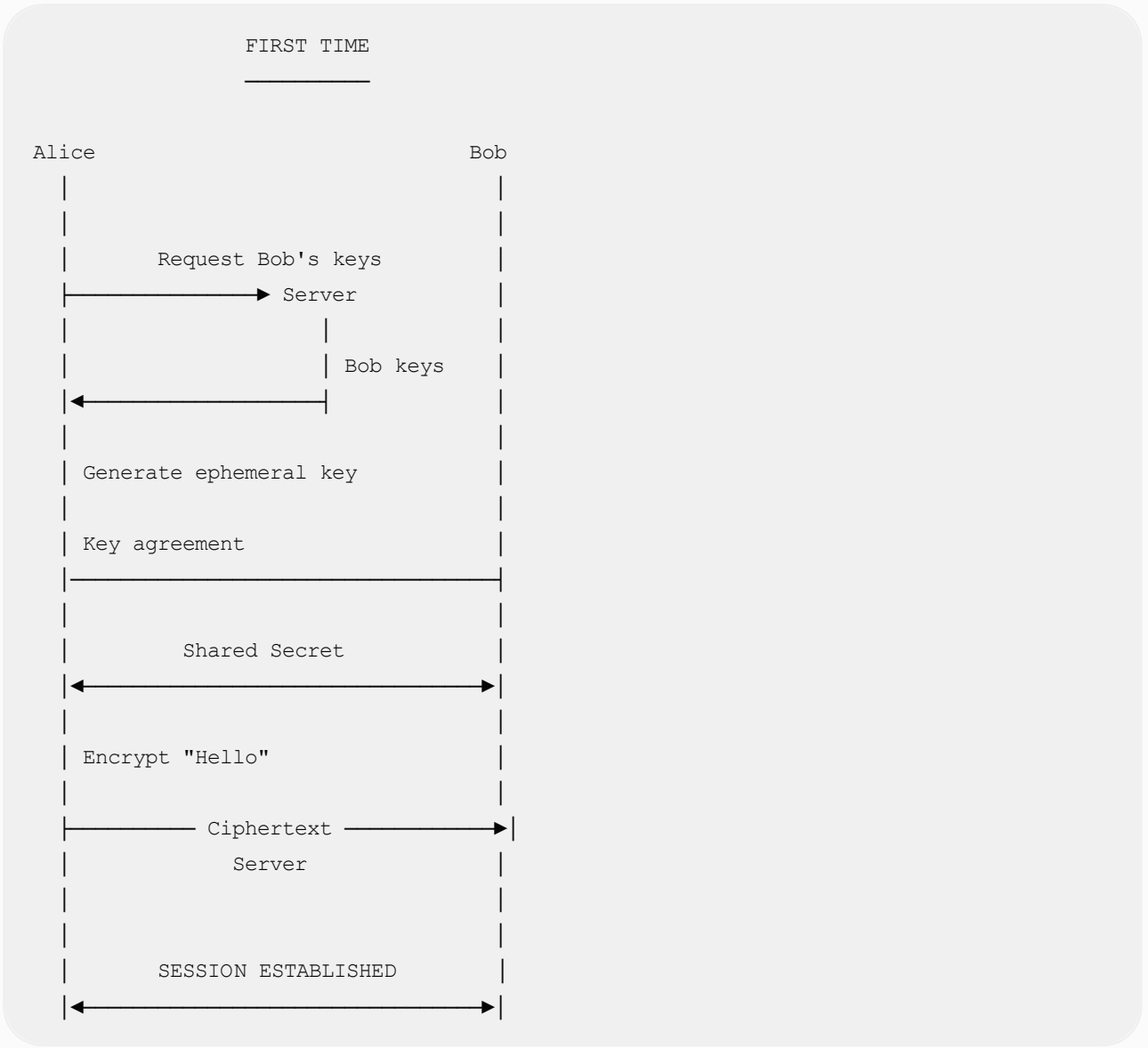
```
Alice
  │
  ├── Derive next message key
  │
  ├── Encrypt message
  │
  ↓
Server
  │
  ↓
Bob
  │
  └── Derive corresponding message key
      ↓
    Decrypt
```

No new OTPK for every message.

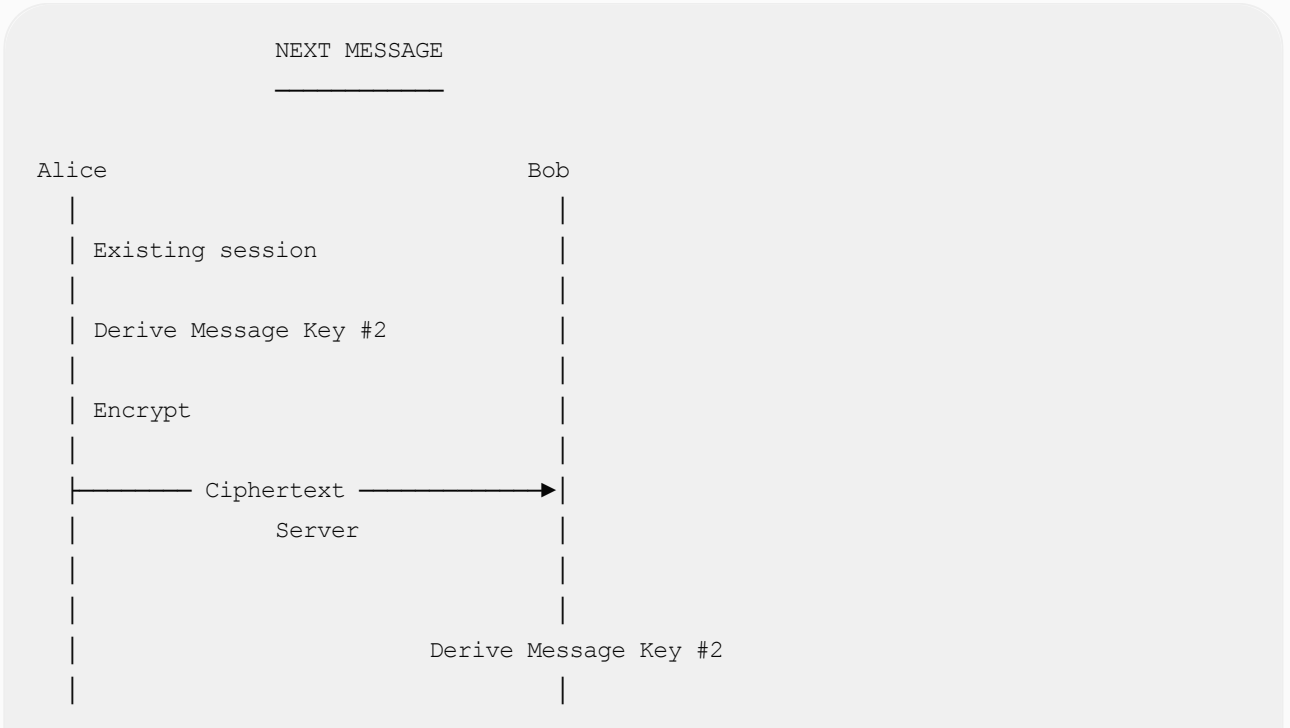
---

18. Put everything together

Here's the complete lifecycle:



Then:

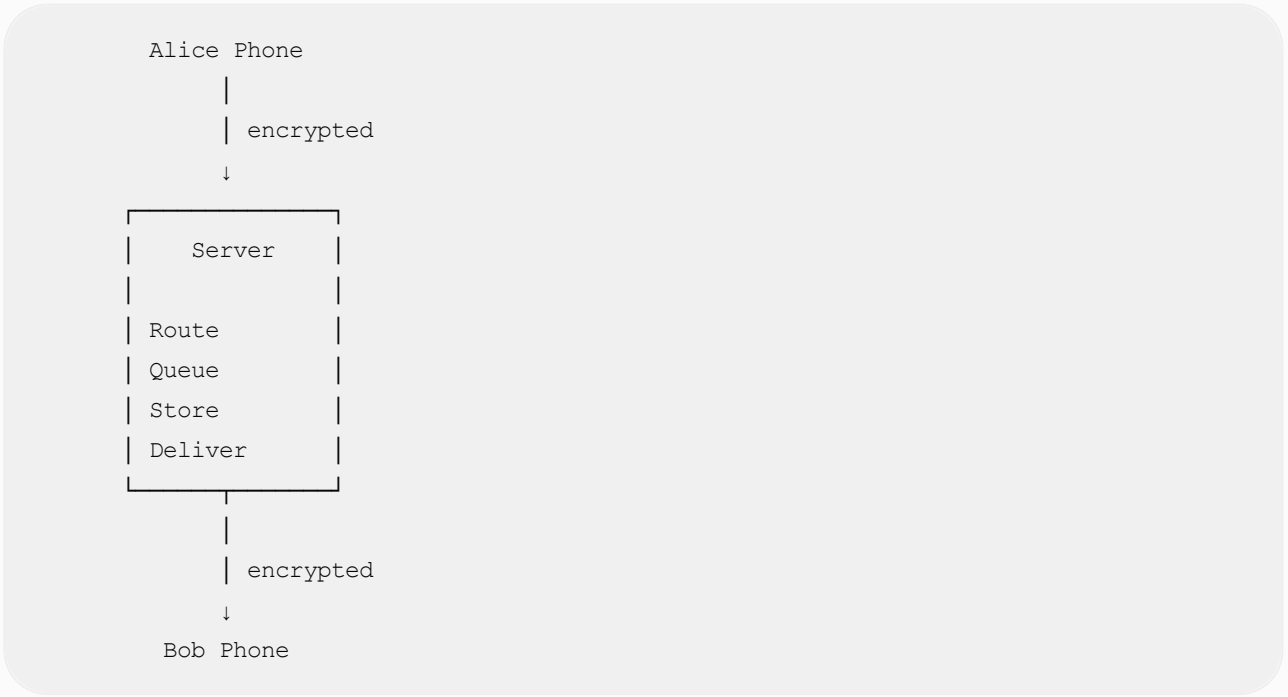


```
   |                        Decrypt
   |                                 |
```

---

19. And where does the server fit?

The server is essentially a **mailbox + router**, not the place where encryption/decryption happens.



The server can generally know things like:

```
Alice sent something to Bob
Time = 10:32
Message size = X
Bob received it
```

But it shouldn't be able to know:

```
"Hello Bob, how are you?"
```

because that plaintext exists only on the endpoints.

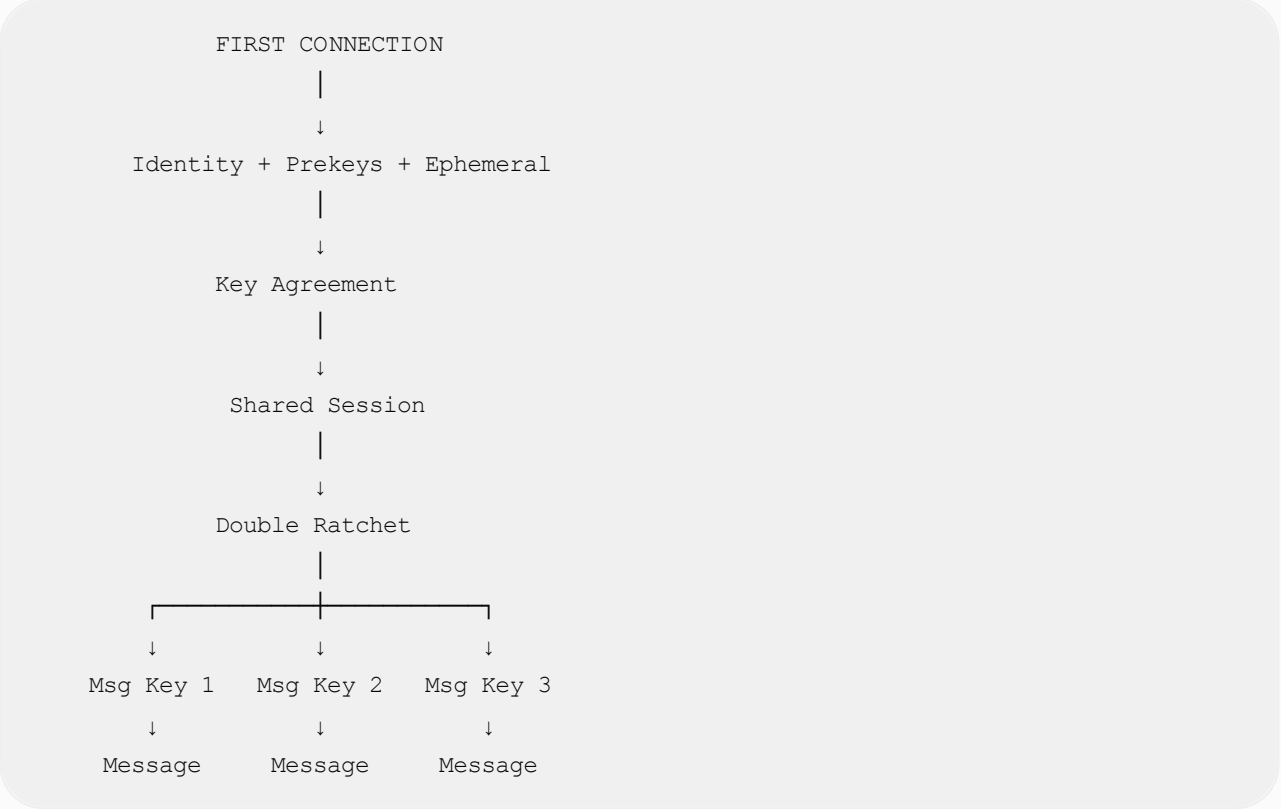
One final distinction

You can think of the cryptographic pieces as having different jobs:

Key	Purpose	Lifetime
Identity key	"I am Bob"	Long-lived

Key	Purpose	Lifetime
Signed prekey	Help establish a session + authenticate key	Long-lived-ish, rotated
One-time prekey	Extra bootstrap material for an initial session	Single use
Ephemeral key	Temporary key used during key agreement	Short-lived
Session/root keys	Maintain the conversation's cryptographic state	Session lifetime
Message keys	Encrypt individual messages	Very short-lived / one message

So the overall picture is:



Prekeys get you into the conversation.
The session/ratchet keeps the conversation going.

That distinction is the key to understanding how E2EE messaging scales to millions of users.

Am I correct in saying that Alice uses the shared secret to create the cipher text? Encrypt(Message, sharedSecret)?

Conceptually yes, but not quite directly. Your mental model is close:

```
ciphertext = Encrypt(message, sharedSecret)
```

But in a real Signal-style protocol, the **shared secret is not normally used directly as the encryption key**.

Instead:

```
                    Initial Key Agreement
                           |
                           ↓
                     Shared Secret
                           |
                           ↓
                    Key Derivation
                         (KDF)
                           |
                           ↓
                    Message Key #1
                           |
                           ↓
            Encrypt(Message, MessageKey1)
                           |
                           ↓
                       Ciphertext
```

Why the extra step?

A shared secret produced by Diffie-Hellman is raw key material. You generally feed it into a **KDF (Key Derivation Function)** to produce cryptographically suitable keys.

For example, conceptually:

```
sharedSecret
     +
context / salt / protocol state
     |
     ↓
    KDF
     |
```

Then:

```
ciphertext = AEAD_Encrypt(
    key = messageKey,
    plaintext = "Hello Bob",
    nonce = ...
)
```

For example, an AEAD algorithm could be **AES-GCM** or **ChaCha20-Poly1305**.

And this becomes more important with Double Ratchet

You don't normally do:

```
Encrypt(message1, sharedSecret)
Encrypt(message2, sharedSecret)
Encrypt(message3, sharedSecret)
```

Instead:

```
Initial shared secret
        |
        ↓
   Root/Chain Keys
        |
        ↓
 ┌──────┼──────┬──────┐
 ↓      ↓      ↓      ↓
MK1    MK2    MK3    MK4
 ↓      ↓      ↓      ↓
M1     M2     M3     M4
```

So:

```
Ciphertext1 = Encrypt(Message1, MK1)
Ciphertext2 = Encrypt(Message2, MK2)
Ciphertext3 = Encrypt(Message3, MK3)
```

This is a **much better mental model**:

> **The initial shared secret establishes the cryptographic session. The ratchet derives fresh message keys from that session, and those message keys encrypt the actual messages.**

And Bob performs the corresponding derivation, so he gets the same `MK1`, `MK2`, etc. and can decrypt the messages.

What is this Root/Chain Key? How do both of them have it?

Exactly — this is the next important piece. Let's build it from the **shared secret** you already understand.

1. First, Alice and Bob get a shared secret

After the initial key exchange:

```
Alice                          Bob
-----                          ---

Key agreement                  Key agreement
      |                              |
      └────────── SAME SECRET ───────┘

             Shared Secret
                  = X
```

Neither Alice nor Bob sent `X` to the other.

They independently calculated the same value.

---

2. What is the Root Key?

The **Root Key** is essentially the starting point for the ongoing conversation's key-derivation process.

Very simplified:

```
Shared Secret
      |
      ↓
   Root Key
      |
      ↓
   Chain Keys
```

Think of the Root Key as the **master starting point for the ratchet**.

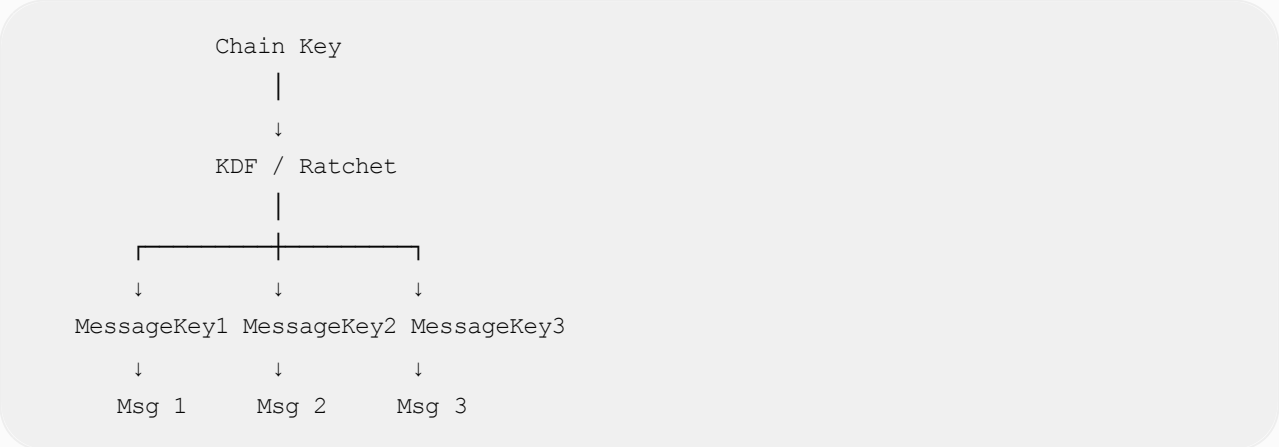
It is **not normally used directly to encrypt messages**.

---

3. What is a Chain Key?

A **Chain Key** is used to generate individual message keys.

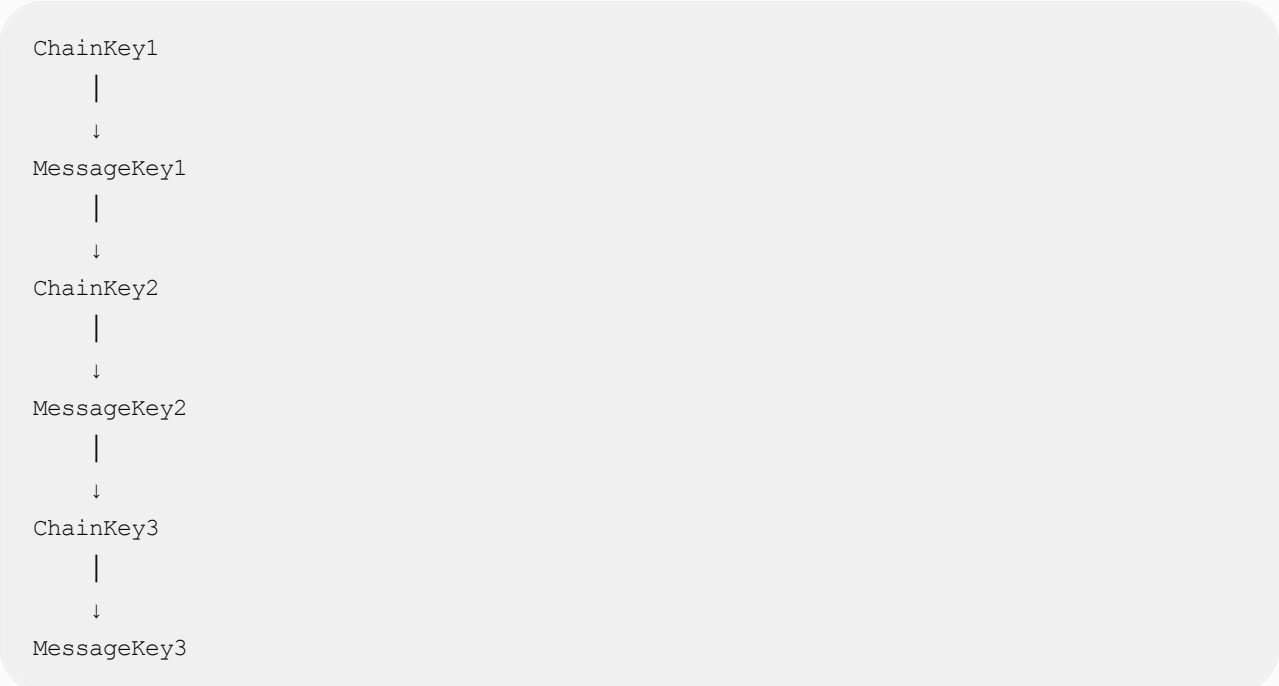
For example:



So:

```
Chain Key → Message Key → Encrypt one message
```

Then the chain advances:



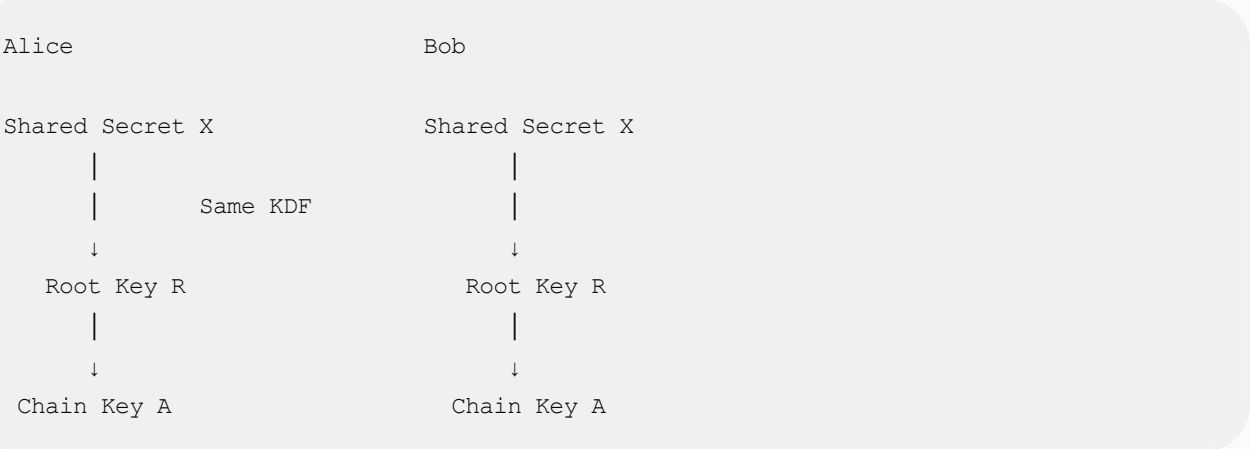
The exact Signal Double Ratchet construction has separate sending/receiving chains and additional DH ratchet steps, but this is the right mental model.

4. But how do Alice and Bob BOTH get the Chain Key?

This is the really interesting part.

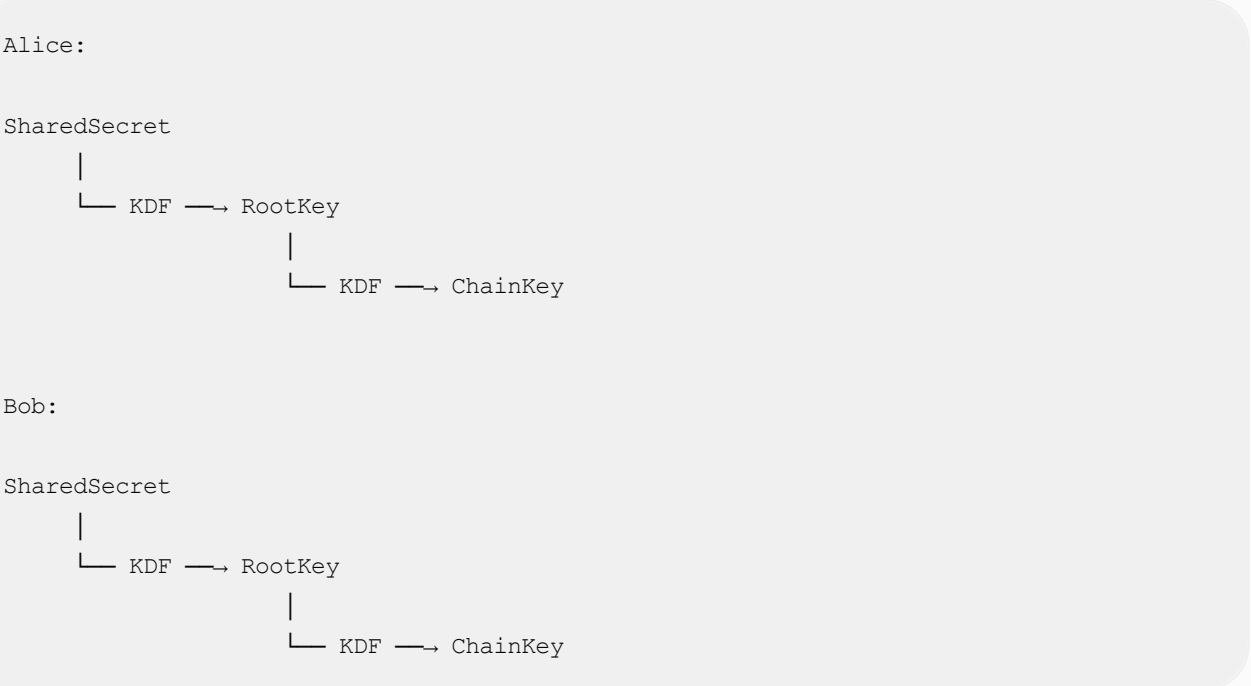
They **don't send the Chain Key to each other**.

Remember:



Because they start with the **same cryptographic material** and perform the same deterministic derivation, they can independently arrive at the same keys.

Conceptually:



Therefore:

```
Alice ChainKey == Bob ChainKey
```

No one transmitted the Chain Key.

5. Then Alice sends Message 1

Alice has:

```
ChainKey1
```

She derives:

```
ChainKey1
    |
    ↓
 KDF
    |
    ↓
MessageKey1
```

Then:

```
Encrypt(
    "Hello Bob",
    MessageKey1
)
```

Alice sends the ciphertext.

Bob already has the corresponding chain state.

He derives the same:

```
ChainKey1
    |
    ↓
 KDF
    |
    ↓
MessageKey1
```

Therefore Bob can decrypt.

---

6. Now comes the clever part: the Chain Key changes

After using `ChainKey1`, Alice doesn't keep using it.

She advances:

```
ChainKey1
    |
    ↓
KDF
    |
    ├──────────→ MessageKey1
    |
    ↓
ChainKey2
```

Then:

```
ChainKey2
    |
    ↓
KDF
    |
    ├──────────→ MessageKey2
    |
    ↓
ChainKey3
```

So:

```
Message 1 → MessageKey1
Message 2 → MessageKey2
Message 3 → MessageKey3
Message 4 → MessageKey4
```

This is the **ratcheting** idea.

---

7. Why not just use the same shared secret for everything?

Suppose we did this:

```
SharedSecret
     |
     ↓
MessageKey
     |
     ├── Message 1
     ├── Message 2
     ├── Message 3
     ├── Message 4
     └── ...
```

Then the same encryption key might protect thousands of messages.

That's undesirable.

Instead:

```
Shared Secret
      |
      ↓
    Root
      |
      ↓
   Chain 1
      |
      ├── MessageKey1
      ├── MessageKey2
      ├── MessageKey3
      └── MessageKey4
```

Each message gets a fresh key.

---

8. And this is where Double Ratchet gets really interesting

So far we've only discussed **one direction**.

In reality, Alice and Bob have separate chains:

```
            Alice                          Bob
              |                              |
              |                              |
       Alice Sending Chain            Bob Sending Chain
              |                              |
              ↓                              ↓
       Alice → Bob                    Bob → Alice
```

Conceptually:

```
Alice's sending chain
    ↓
A-MK1 → Message
    ↓
A-MK2 → Message
    ↓
A-MK3 → Message


Bob's sending chain
    ↓
B-MK1 → Message
    ↓
B-MK2 → Message
    ↓
B-MK3 → Message
```

So when Alice sends:

```
Alice → Bob
```

Alice uses her **sending chain** and Bob uses his corresponding **receiving chain**.

When Bob replies:

```
Bob → Alice
```

Bob uses his sending chain and Alice uses her corresponding receiving chain.

---

9. And then there is the SECOND ratchet

This is why it's called **Double Ratchet**.

The chain-key ratchet gives you:

```
Message 1 → Key 1
Message 2 → Key 2
Message 3 → Key 3
...
```

But periodically the parties also perform a new **Diffie-Hellman key exchange**.

Conceptually:

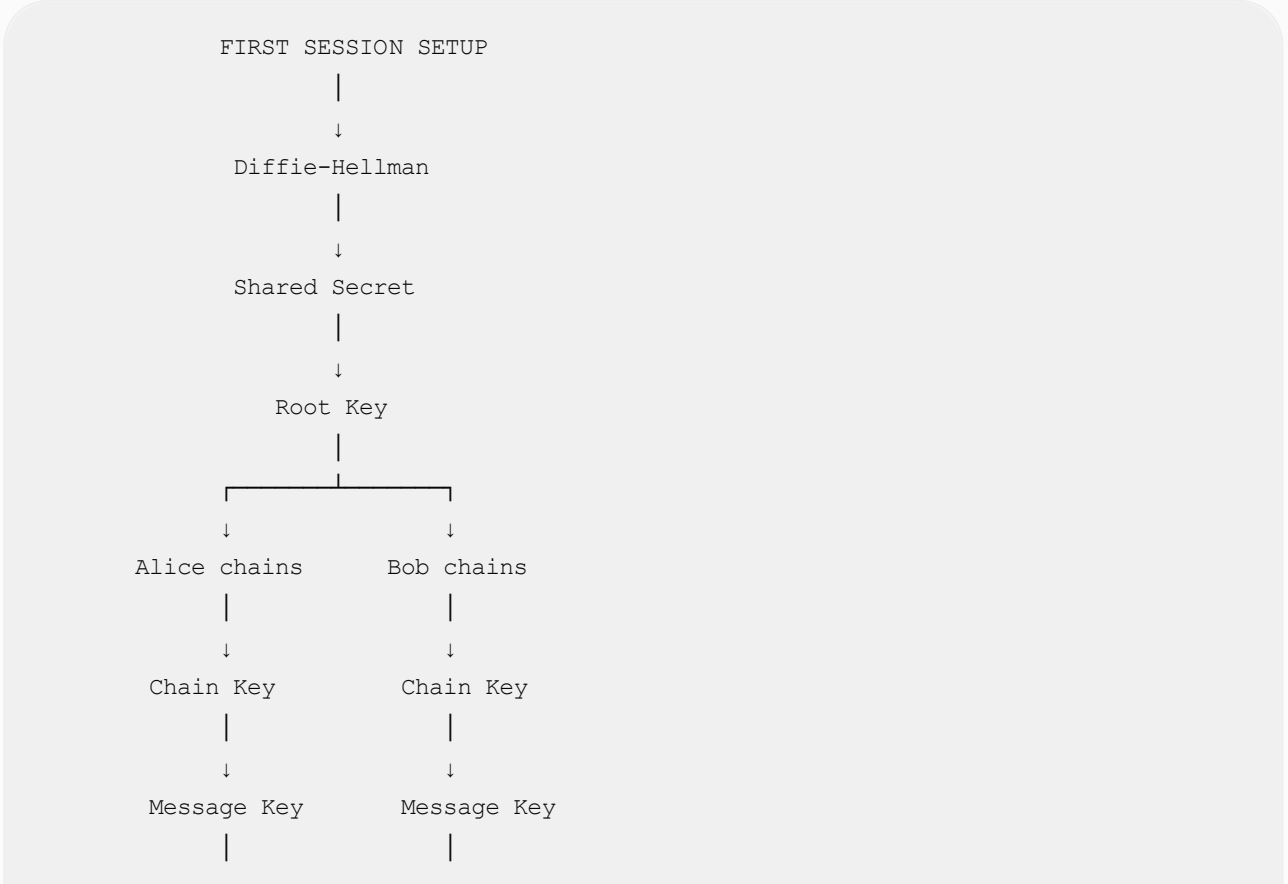
That's the second ratchet.

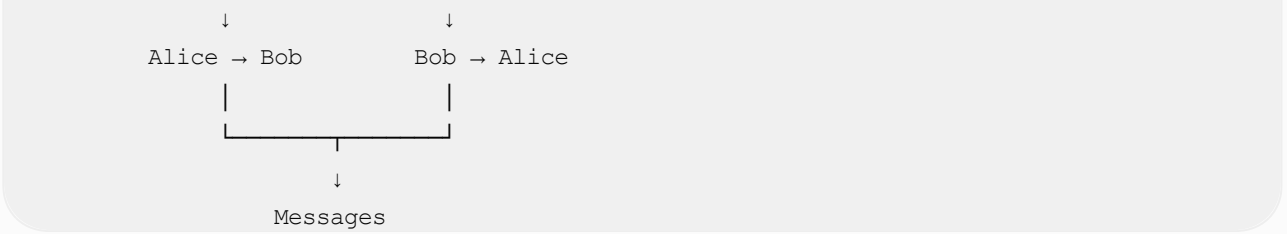
It means the cryptographic state keeps moving forward rather than staying fixed.

---

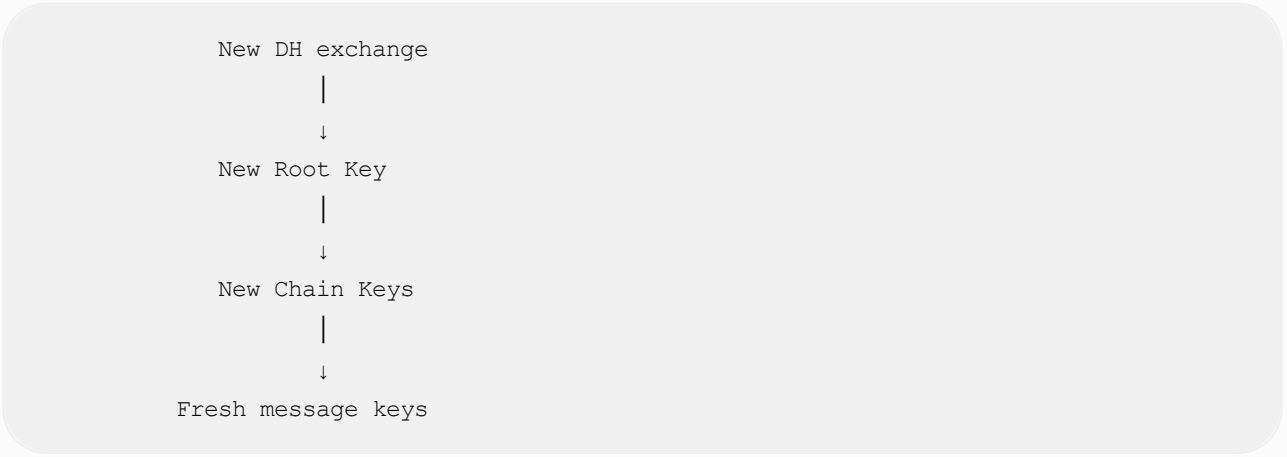
10. The whole picture

Now connect everything we've discussed:





And periodically:



The simplest mental model

Think of it like this:

Shared Secret
→ establishes the cryptographic relationship

Root Key
→ the master starting point for the ratchet

Chain Key
→ generates keys for a sequence of messages

Message Key
→ encrypts one individual message

So:

> **Alice and Bob don't exchange Root Keys or Chain Keys. They independently derive them from shared cryptographic state.**

And that's the beautiful part of the design: **the server is transporting ciphertext, while Alice and Bob independently maintain synchronized cryptographic state.**

So, they generate multiple shared secret across their lifetime of conversation?

Yes — **but with an important distinction**.

They can generate **multiple DH shared secrets during the lifetime of a conversation**, because the Double Ratchet periodically performs a new Diffie-Hellman exchange.

But **not for every message**.

Think of it in two levels

1. Most messages → new message key

```
Shared DH state
      ↓
Root Key
      ↓
Chain Key
   ↓   ↓   ↓
  MK1 MK2 MK3 ...
   ↓   ↓   ↓
 Msg1 Msg2 Msg3
```

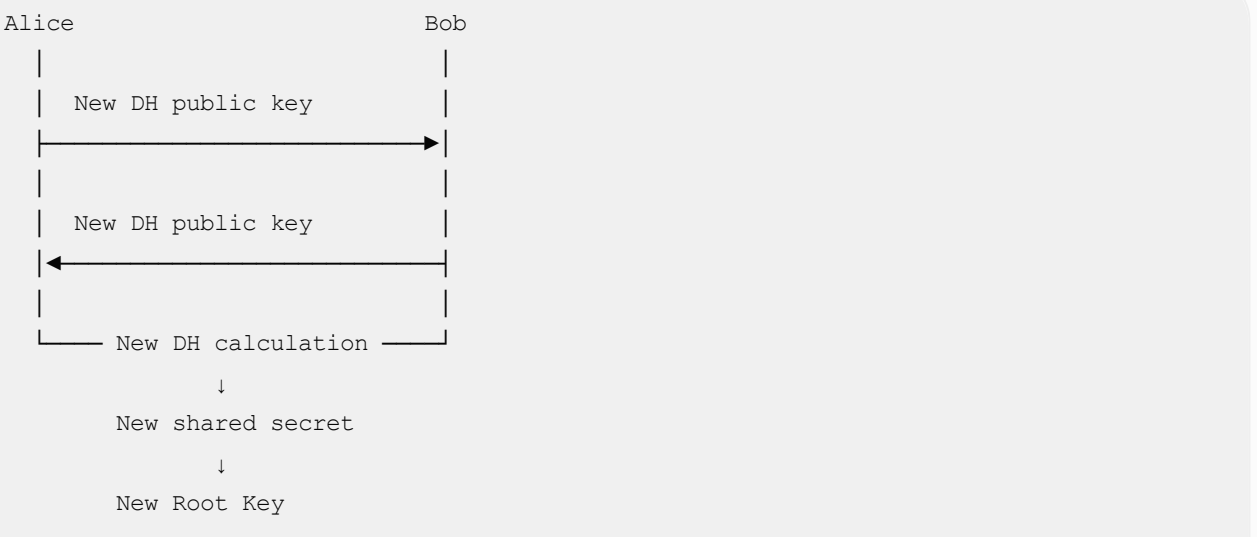
For messages 1, 2, 3, etc., they're just advancing the **symmetric key chain**.

No new DH shared secret is needed for every message.

---

2. Periodically → new DH exchange

At some point, Alice and Bob exchange fresh ephemeral/DH public keys:



```
            ↓
      New Chain Keys
            ↓
      New Message Keys
```

So over a long conversation you might conceptually have:

```
Conversation
│
├── DH Shared Secret #1
│       └── Messages 1–50
│
├── DH Shared Secret #2
│       └── Messages 51–120
│
├── DH Shared Secret #3
│       └── Messages 121–200
│
└── ...
```

The exact boundaries aren't simply "every 50 messages"; the actual Double Ratchet advances based on the parties' sending/receiving ratchet steps.

Why do this?

This is a major security feature.

If an attacker somehow gets access to the current cryptographic state, the **new DH ratchet steps can move the conversation into fresh cryptographic territory**.

So you have two mechanisms working together:

```
            DOUBLE RATCHET
                  │
        ┌─────────┴─────────┐
        ↓                   ↓
 Symmetric ratchet      DH ratchet
        │                   │
        ↓                   ↓
New message key        New shared secret
for every message      periodically
        │                   │
        └─────────┬─────────┘
                  ↓
             New root/
             chain state
```

So your statement is basically correct:

> **During a long-lived conversation, Alice and Bob can establish multiple fresh DH-derived shared secrets over time, while individual messages between those DH ratchet steps use keys derived from the current symmetric chain.**

That's the key idea behind **Double** Ratchet: one ratchet for message keys, another based on fresh DH exchanges.

Give me an example in golang for this flow.

Yes. Let's build a **small Go simulation** of the flow we've been discussing.

One caveat: this is an educational implementation of the **key lifecycle**, not a production Signal Protocol implementation. A real implementation needs the full Double Ratchet specification, authenticated key exchange, replay protection, skipped-message handling, secure key storage, etc.

We'll model:

```
FIRST TIME

Alice                                Bob
  |                                   |
  |——— public key material ————————→|
  |                                   |
  |   DH / shared secret              |
  |←——————————————————————————————→|
  |                                   |
  |       Root Key                    |
  |          ↓                        |
  |       Chain Key                   |
  |          ↓                        |
  |       Message Key                 |
  |          ↓                        |
  |       Encrypt                     |
  |          ↓                        |
  |——————— ciphertext ————————————→|
```

Then subsequent messages:

```
Chain Key
    |
    ├── Message Key 1 → Message 1
    |
```

```
    ├── Message Key 2 → Message 2
    │
    └── Message Key 3 → Message 3
```

And periodically:

```
New DH exchange
      ↓
New shared secret
      ↓
New Root Key
      ↓
New Chain Key
```

---

1. Start with X25519

Go's `crypto/ecdh` package gives us X25519.

We'll use:

```
    Go

ecdh.X25519()
```

First, let's create a helper to generate a key pair.

```
    Go

package main

import (
    "crypto/ecdh"
    "crypto/rand"
    "crypto/sha256"
    "encoding/hex"
    "fmt"
)

func generateKeyPair() (*ecdh.PrivateKey, *ecdh.PublicKey, error) {
    privateKey, err := ecdh.X25519().GenerateKey(rand.Reader)
    if err != nil {
        return nil, nil, err
    }

    return privateKey, privateKey.PublicKey(), nil
}
```

Now Alice can generate an ephemeral key:

```go
alicePrivate, alicePublic, _ := generateKeyPair()
```

Bob can have his own key:

```go
bobPrivate, bobPublic, _ := generateKeyPair()
```

---

2. Generate the shared secret

This is the Diffie-Hellman operation.

Alice does:

```go
aliceSharedSecret, err := alicePrivate.ECDH(bobPublic)
if err != nil {
    panic(err)
}
```

Bob independently does:

```go
bobSharedSecret, err := bobPrivate.ECDH(alicePublic)
if err != nil {
    panic(err)
}
```

And:

```go
fmt.Println(hex.EncodeToString(aliceSharedSecret))
fmt.Println(hex.EncodeToString(bobSharedSecret))
```

will produce the same value.

Conceptually:

```
Alice:

AlicePrivate + BobPublic
```

```
          ↓
     SharedSecret


Bob:

BobPrivate + AlicePublic
          ↓
     SharedSecret
```

Therefore:

```go
bytes.Equal(aliceSharedSecret, bobSharedSecret)
```

is `true`.

---

3. But we don't use SharedSecret directly

Now we introduce the **Root Key**.

For this educational example, we'll derive it using SHA-256:

```go
func deriveRootKey(sharedSecret []byte) []byte {
    hash := sha256.New()

    hash.Write([]byte("root-key"))
    hash.Write(sharedSecret)

    return hash.Sum(nil)
}
```

Then:

```go
aliceRootKey := deriveRootKey(aliceSharedSecret)
bobRootKey := deriveRootKey(bobSharedSecret)
```

Again:

```
Alice                          Bob
  |                             |
  | Shared Secret               | Shared Secret
```

```
|      ↓                        |      ↓
|  deriveRootKey                |  deriveRootKey
|      ↓                        |      ↓
|  Root Key                     |  Root Key
|                               |
└──────── SAME ─────────────────┘
```

---

4. Generate a Chain Key

Now derive a chain key from the root key.

Go

```go
func deriveChainKey(rootKey []byte) []byte {
    hash := sha256.New()

    hash.Write([]byte("chain-key"))
    hash.Write(rootKey)

    return hash.Sum(nil)
}
```

So:

Go

```go
aliceChainKey := deriveChainKey(aliceRootKey)
bobChainKey := deriveChainKey(bobRootKey)
```

Now both sides have:

```
Root Key
    ↓
Chain Key
```

without ever sending those keys over the network.

---

5. Generate a Message Key

Now we need a key to encrypt **one message**.

Go

```go
func deriveMessageKey(chainKey []byte) []byte {
    hash := sha256.New()
```

```
    hash.Write([]byte("message-key"))
    hash.Write(chainKey)

    return hash.Sum(nil)
}
```

Alice:

```
    Go

aliceMessageKey := deriveMessageKey(aliceChainKey)
```

Bob:

```
    Go

bobMessageKey := deriveMessageKey(bobChainKey)
```

Again:

```
Alice Message Key == Bob Message Key
```

---

6. But we need to advance the chain

We don't want:

```
Message 1 → same key
Message 2 → same key
Message 3 → same key
```

Instead:

```
ChainKey1
   |
   ├──→ MessageKey1
   |
   ↓
ChainKey2
   |
   ├──→ MessageKey2
   |
   ↓
ChainKey3
```

We can implement:

```go
func ratchetChainKey(chainKey []byte) []byte {
    hash := sha256.New()

    hash.Write([]byte("chain-ratchet"))
    hash.Write(chainKey)

    return hash.Sum(nil)
}
```

Now Alice can do:

```go
messageKey1 := deriveMessageKey(aliceChainKey)

aliceChainKey = ratchetChainKey(aliceChainKey)

messageKey2 := deriveMessageKey(aliceChainKey)

aliceChainKey = ratchetChainKey(aliceChainKey)

messageKey3 := deriveMessageKey(aliceChainKey)
```

Bob performs the same operations.

---

7. Now we need actual encryption

For real encryption, let's use AES-GCM.

```go
import (
    "crypto/aes"
    "crypto/cipher"
)
```

Encryption:

```go
func encrypt(key []byte, plaintext []byte) ([]byte, error) {
    block, err := aes.NewCipher(key[:32])
    if err != nil {
        return nil, err
    }
```

```
    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return nil, err
    }

    nonce := make([]byte, gcm.NonceSize())

    if _, err := rand.Read(nonce); err != nil {
        return nil, err
    }

    ciphertext := gcm.Seal(
        nonce,
        nonce,
        plaintext,
        nil,
    )

    return ciphertext, nil
}
```

Notice something important:

```
Message
   +
Message Key
   ↓
AES-GCM
   ↓
Ciphertext
```

Not:

```
Message
   +
Shared Secret
   ↓
Ciphertext
```

---

8. Decryption

Bob needs the same message key:

```
    Go

func decrypt(key []byte, ciphertext []byte) ([]byte, error) {
    block, err := aes.NewCipher(key[:32])
```

```
    if err != nil {
        return nil, err
    }

    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return nil, err
    }

    nonceSize := gcm.NonceSize()

    nonce := ciphertext[:nonceSize]
    data := ciphertext[nonceSize:]

    return gcm.Open(
        nil,
        nonce,
        data,
        nil,
    )
}
```

---

9. Put the first message together

Alice:

Go

```
message := []byte("Hello Bob!")

messageKey := deriveMessageKey(aliceChainKey)

ciphertext, err := encrypt(messageKey, message)
if err != nil {
    panic(err)
}

fmt.Println("Ciphertext:", hex.EncodeToString(ciphertext))
```

Alice sends only the ciphertext through the server:

```
Alice
  |
  | ciphertext
  ↓
Server
  |
  | ciphertext
```

```
↓
```

---

10. Bob decrypts

Bob independently generated the same message key:

```go
messageKey := deriveMessageKey(bobChainKey)

plaintext, err := decrypt(messageKey, ciphertext)
if err != nil {
    panic(err)
}

fmt.Println(string(plaintext))
```

Output:

```
Hello Bob!
```

---

11. Now send message #2

Here's the interesting part.

Alice advances her chain:

```go
aliceChainKey = ratchetChainKey(aliceChainKey)

messageKey2 := deriveMessageKey(aliceChainKey)

ciphertext2, _ := encrypt(
    messageKey2,
    []byte("How are you?"),
)
```

Bob does the same:

```go
bobChainKey = ratchetChainKey(bobChainKey)

messageKey2 := deriveMessageKey(bobChainKey)
```

```
plaintext2, _ := decrypt(
    messageKey2,
    ciphertext2,
)
```

Bob gets:

```
How are you?
```

---

12. The complete simplified program

Here's the core flow in one place:

Go

```go
package main

import (
    "crypto/aes"
    "crypto/cipher"
    "crypto/ecdh"
    "crypto/rand"
    "crypto/sha256"
    "fmt"
)

func generateKeyPair() (*ecdh.PrivateKey, *ecdh.PublicKey, error) {
    privateKey, err := ecdh.X25519().GenerateKey(rand.Reader)
    if err != nil {
        return nil, nil, err
    }

    return privateKey, privateKey.PublicKey(), nil
}

func deriveRootKey(sharedSecret []byte) []byte {
    h := sha256.New()
    h.Write([]byte("root-key"))
    h.Write(sharedSecret)
    return h.Sum(nil)
}

func deriveChainKey(rootKey []byte) []byte {
    h := sha256.New()
    h.Write([]byte("chain-key"))
    h.Write(rootKey)
    return h.Sum(nil)
```

```
}

func deriveMessageKey(chainKey []byte) []byte {
    h := sha256.New()
    h.Write([]byte("message-key"))
    h.Write(chainKey)
    return h.Sum(nil)
}

func ratchetChainKey(chainKey []byte) []byte {
    h := sha256.New()
    h.Write([]byte("chain-ratchet"))
    h.Write(chainKey)
    return h.Sum(nil)
}

func encrypt(key []byte, plaintext []byte) ([]byte, error) {
    block, err := aes.NewCipher(key)
    if err != nil {
        return nil, err
    }

    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return nil, err
    }

    nonce := make([]byte, gcm.NonceSize())

    if _, err := rand.Read(nonce); err != nil {
        return nil, err
    }

    return gcm.Seal(nonce, nonce, plaintext, nil), nil
}

func decrypt(key []byte, ciphertext []byte) ([]byte, error) {
    block, err := aes.NewCipher(key)
    if err != nil {
        return nil, err
    }

    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return nil, err
    }

    nonceSize := gcm.NonceSize()

    return gcm.Open(
        nil,
        ciphertext[:nonceSize],
```

```
        ciphertext[nonceSize:],
        nil,
    )
}

func main() {

    // ============================================================
    // 1. INITIAL KEY EXCHANGE
    // ============================================================

    alicePrivate, alicePublic, _ := generateKeyPair()
    bobPrivate, bobPublic, _ := generateKeyPair()

    // Alice calculates shared secret
    aliceSharedSecret, _ := alicePrivate.ECDH(bobPublic)

    // Bob calculates shared secret
    bobSharedSecret, _ := bobPrivate.ECDH(alicePublic)

    fmt.Println(
        "Shared secrets equal:",
        string(aliceSharedSecret) == string(bobSharedSecret),
    )

    // ============================================================
    // 2. DERIVE ROOT KEY
    // ============================================================

    aliceRootKey := deriveRootKey(aliceSharedSecret)
    bobRootKey := deriveRootKey(bobSharedSecret)

    // ============================================================
    // 3. DERIVE CHAIN KEY
    // ============================================================

    aliceChainKey := deriveChainKey(aliceRootKey)
    bobChainKey := deriveChainKey(bobRootKey)

    // ============================================================
    // 4. MESSAGE 1
    // ============================================================

    aliceMessageKey := deriveMessageKey(aliceChainKey)

    ciphertext, _ := encrypt(
        aliceMessageKey,
        []byte("Hello Bob!"),
    )

    // Bob independently derives same message key
    bobMessageKey := deriveMessageKey(bobChainKey)
```

```
    plaintext, _ := decrypt(
        bobMessageKey,
        ciphertext,
    )

    fmt.Println("Bob received:", string(plaintext))

    // ============================================================
    // 5. RATCHET
    // ============================================================

    aliceChainKey = ratchetChainKey(aliceChainKey)
    bobChainKey = ratchetChainKey(bobChainKey)

    // ============================================================
    // 6. MESSAGE 2
    // ============================================================

    aliceMessageKey = deriveMessageKey(aliceChainKey)

    ciphertext, _ = encrypt(
        aliceMessageKey,
        []byte("How are you?"),
    )

    bobMessageKey = deriveMessageKey(bobChainKey)

    plaintext, _ = decrypt(
        bobMessageKey,
        ciphertext,
    )

    fmt.Println("Bob received:", string(plaintext))
```

The output is roughly:

```
Shared secrets equal: true
Bob received: Hello Bob!
Bob received: How are you?
```

---

13. But our example is still missing the "Double" in Double Ratchet

What we've implemented so far is basically the **symmetric-key ratchet**:

```
Shared Secret
      ↓
```

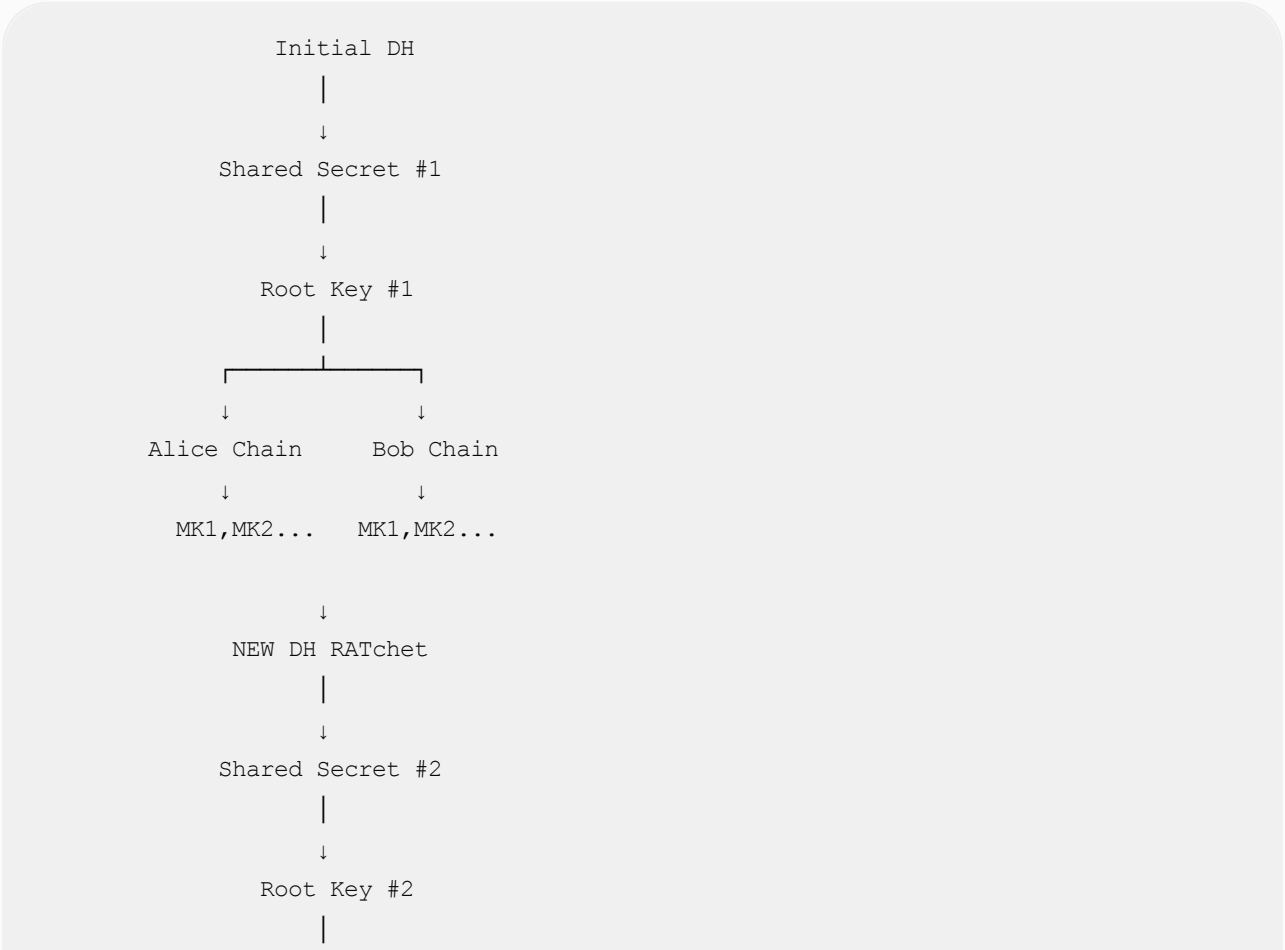
```
   Root Key
      ↓
  Chain Key
      ↓
Message Key 1
      ↓
Message Key 2
      ↓
Message Key 3
```

The **second ratchet** is the DH ratchet.

Conceptually, after some communication:

```
Alice generates new DH key
          +
Bob's current DH public key
          ↓
     NEW shared secret
          ↓
      NEW Root Key
          ↓
      NEW Chain Keys
          ↓
      NEW Message Keys
```

So the complete system becomes:



```
                ┌──────┴──────┐
                ↓             ↓
        New Alice Chain  New Bob Chain
```

And later:

```
Shared Secret #1
        ↓
Shared Secret #2
        ↓
Shared Secret #3
        ↓
Shared Secret #4
        ↓
...
```

while individual messages continuously get fresh message keys in between.

The key thing I want you to notice

You asked earlier:

> "How do both of them have the Root/Chain Key?"

The Go example makes the answer concrete:

Go

```go
// Alice
aliceSharedSecret, _ := alicePrivate.ECDH(bobPublic)

// Bob
bobSharedSecret, _ := bobPrivate.ECDH(alicePublic)
```

Both get:

```
SAME shared secret
```

Then both execute the same deterministic functions:

Go

```go
RootKey  = KDF(SharedSecret)
ChainKey = KDF(RootKey)
MsgKey   = KDF(ChainKey)
```

Therefore:

Nothing except the public keys and ciphertext needs to cross the server.

That's the fundamental trick that makes E2E encryption work.